

MANUAL DO UTILIZADOR

Maprisma

& AI Agents Map

Os seus agentes de IA, modelos locais, servidores MCP e skills num só mapa

Maprisma 0.12.0 · AI Agents Map 0.26.1
outubro 2026

Sobre este manual

Este manual descreve uma aplicação que existe em duas edições:

Edição	Maprisma	AI Agents Map
O que é	Uma app independente GTK 4 / libadwaita para qualquer ambiente de trabalho Linux	Um plugin para o ambiente de trabalho Omarchy, aberto a partir de um botão na barra
Instala-se a partir de	AUR (`maprisma-bin`) ou AppImage	O Omarchy Plugin Marketplace
Aparência	Segue o modo claro/escuro e a cor de destaque do sistema	As cores do tema Omarchy ou a aparência Adwaita do GNOME
Pasta das definições	`~/.config/maprisma/`	`~/.config/omarchy/ai-agents-map/`

Tudo o resto — Configuração, o mapa, ligar desenhando, os LLMs locais, os modelos, os servidores MCP e as skills — é igual em ambas. O que só se aplica a uma edição está marcado com MAPRISMA ou OMARCHY.

Convenções

Adicionar	Um botão, separador ou item de menu da janela.
Definições › **Aparência**	Um caminho pela interface: abra as definições e depois **Aparência**.
`~/.agents/skills`	Um ficheiro, uma pasta ou um comando. `~` é a sua pasta pessoal.
Ctrl+,	Um atalho de teclado.

> **NOTA** As notas dão contexto útil.

> **DICA** As dicas mostram um caminho mais rápido ou um bom hábito.

> **ATENÇÃO** As caixas de atenção assinalam algo difícil de desfazer.

Índice

1 **Introdução** 1
- 1.1 O que faz 1
- 1.2 Ideias-chave 2
- 1.3 Nada muda sem pré-visualização 2

2 **Instalação** 3
- 2.1 Instalar o Maprisma 3
- 2.2 Primeiro arranque 3
- 2.3 Remover a app 4

3 **A janela num relance** 5
- 3.1 Configuração 6
- 3.2 Escolher o que o mapa mostra 8
- 3.3 Trabalhar com o mapa 9
- 3.4 Problemas por resolver 10
- 3.5 Os problemas por resolver na pré-visualização Flutter 10
- 3.6 Atualizações e sugestões 11
- 3.7 Sugestões de complementos na pré-visualização Flutter 11
- 3.8 Cartões inteligentes no mapa 11

4 **Agentes** 12
- 4.1 Detalhes de um agente 12
- 4.2 Iniciar e parar agentes 13
- 4.3 Tráfego de rede 14
- 4.4 Subagentes 14
- 4.5 Agentes que trabalham juntos na pré-visualização Flutter 15
- 4.6 Custos na pré-visualização Flutter 15
- 4.7 Detalhes na pré-visualização Flutter 16
- 4.8 Separadores de terminal, caixa de uso e modelo inicial na pré-visualização Flutter 16
- 4.9 Configurar… 16
- 4.10 Entrar 16
- 4.11 Chaves de API na pré-visualização Flutter 17
- 4.12 Atualizar agentes 18
- 4.13 Atualizar um agente em execução na pré-visualização Flutter 18
- 4.14 Agentes que a app pode instalar 18
- 4.15 Agentes novos 18

5 **LLMs locais** 19
- 5.1 Instalar um LLM local 19
- 5.2 Instalar na pré-visualização Flutter 19
- 5.3 Iniciar e parar com os agentes 20
- 5.4 Detalhes de um LLM local 20
- 5.5 Desempenho 20
- 5.6 O desempenho na pré-visualização Flutter 21
- 5.7 Memória da GPU e libertar memória da GPU 21
- 5.8 Modelos ao mesmo tempo 22
- 5.9 Modelos em simultâneo na pré-visualização Flutter 22
- 5.10 Usado por outras apps 22

6 **Modelos** 24
- 6.1 Obter do Hugging Face 24
- 6.2 Transferências interrompidas na pré-visualização Flutter 25
- 6.3 Perguntar a um agente que modelo serve 25
- 6.4 Instalados 25
- 6.5 Tamanho do contexto 25
- 6.6 Neste PC 26
- 6.7 Mover e eliminar modelos 26
- 6.8 Uma pasta dos modelos 26

7 **Ligar desenhando** 27
- 7.1 Agente ↔ LLM local 27
- 7.2 Agente ↔ servidor MCP 27
- 7.3 Agente ↔ pasta de skills 27
- 7.4 Encontrado neste PC 27
- 7.5 Ligações locais na pré-visualização Flutter 28
- 7.6 Encontrados neste PC na pré-visualização Flutter 28
- 7.7 Quando uma ligação falha 29

8 **Servidores MCP** 30
- 8.1 De onde vêm os servidores 30
- 8.2 Configurar um servidor 31
- 8.3 Os servidores MCP na pré-visualização Flutter 31
- 8.4 Gerir servidores 31

9 **Skills** 33
- 9.1 Encontrar skills 33
- 9.2 Skills da web na pré-visualização Flutter 33
- 9.3 Pastas de skills 33

10 **Definições e ajuda** 34
- 10.1 Temas e luzes do mapa Flutter 34
- 10.2 A janela flutuante sob a barra no Omarchy 34
- 10.3 Dados de utilização e cantos do Omarchy na pré-visualização Flutter 34
- 10.4 Definições 35
- 10.5 Limites a vermelho na pré-visualização Flutter 36
- 10.6 Alterações recentes e Desfazer após um reinício 37
- 10.7 Ajuda 37
- 10.8 A ajuda na pré-visualização Flutter 37
- 10.9 Espaço em disco 37

11 **Smart Check** 39
- 11.1 Resumo da configuração do PC 40

11.2 Atualizações encontradas 40
11.3 Enquanto um agente trabalha 41
11.4 O ajudante local 41
11.5 Verificações com IA e o ajudante local na pré-visualização Flutter 41
11.6 Protocolo 42
11.7 Estatísticas de tokens 42
12 Cartões de grupo e complementos no mapa 43
12.1 Expandir uma categoria 43
12.2 Gerir complementos 43
12.3 Primeira utilização e funções existentes 43
13 Editar o mapa de IA Flutter 44
14 Cartões de tarefas no mapa 45
15 Mapa do sistema 46
15.1 Primeira utilização e discos separados 46
15.2 Mapa guardado e atualização 46
15.3 Ligar, mover e desfazer 46
15.4 Análise e ajuda de IA 46
16 Verificação de saúde e segurança 48
16.1 Ler os cartões 48
16.2 Alterar uma definição 49
16.3 A segurança na pré-visualização Flutter 49
16.4 O estado na pré-visualização Flutter 49
16.5 O que é verificado em cada agente 50
16.6 Controlar o risco 50
16.7 As palavras explicadas 51
17 Complementos 53
17.1 Configurar o Maprisma 53
17.2 A página de complementos 54
17.3 Instalar, ativar e desativar, atualizar, remover 55
17.4 Níveis de privacidade 56
17.5 Tipos de complementos 56
17.6 Um agente novo no seu computador 57
17.7 Criar um complemento 57
18 Privacidade e segurança 58
19 Resolução de problemas 59
A Referência 60
A.1 Atalhos de teclado 60
A.2 Uso com o teclado na pré-visualização Flutter 60
A.3 Ficheiros e pastas 60
A.4 Glossário 61

CAPÍTULO 1

Introdução

O Maprisma mostra a configuração de IA do seu computador como cartões e como mapa: que agentes de programação estão instalados, que modelos de linguagem usam — serviços na nuvem e motores locais —, que modelos estes servem e que servidores MCP e skills tem cada agente. E permite-lhe mudar essa configuração desenhando linhas.

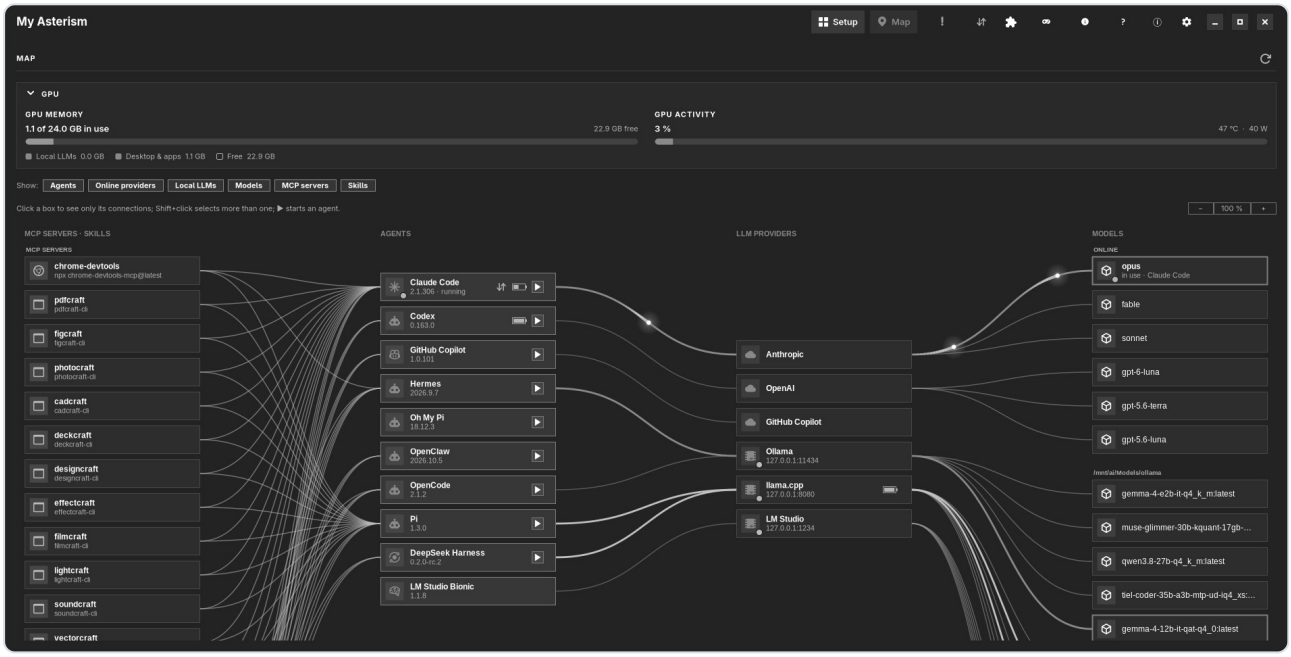


Figura 1 · A janela principal: o mapa em toda a largura, por cima o cartão GPU, no cabeçalho o ! dos problemas por resolver.

1.1 O que faz

- **Agentes de programação** — Claude Code, Codex, GitHub Copilot, Cursor CLI, Gemini, OpenCode, Pi, Hermes e outros: a versão, o plano, os limites e o uso, e o que falta a cada um, com o comando que o resolve.
- **LLMs locais** — llama.cpp, Ollama, LM Studio, vLLM, Jan e KoboldCpp: instalá-los, descarregar modelos, definir o tamanho de contexto que cabe na sua placa gráfica e iniciá-los com os agentes que os usam.
- **Servidores MCP e skills** — encontrados no seu computador ou na web, com servidores recomendados e uma pesquisa no registo MCP oficial.
- **Problemas por resolver** — um ! no cabeçalho quando há algo a corrigir, cada problema com o caminho até onde se corrige, e para que serve a sua configuração.

1.2 Ideias-chave

Agente	Um programa com o qual fala e que escreve código ou faz tarefas, como o Claude Code.
Fornecedor de LLM	Onde corre o modelo de um agente: um serviço na nuvem (Anthropic, OpenAI, …) ou um motor de LLM local neste computador (llama.cpp, Ollama, …).
Modelo	O modelo de linguagem propriamente dito. Os modelos locais são ficheiros, quase sempre em formato GGUF.
Servidor MCP	Um pequeno programa ou serviço web que dá ferramentas extra aos agentes — ler ficheiros, pesquisar na web, controlar um navegador. MCP é o Model Context Protocol.
Skill	Uma pasta com instruções (um `SKILL.md`) e por vezes scripts, que ensina a um agente uma tarefa concreta.

1.3 Nada muda sem pré-visualização

Cada alteração às definições de um agente é mostrada primeiro, linha a linha, antes de se escrever o que quer que seja. O ficheiro antigo é guardado como cópia de segurança, e a barra que aparece depois da alteração tem **Desfazer**. A app em si nunca corre com permissões de administrador; só a instalação de um agente que vem como pacote do sistema pede a sua palavra-passe, num terminal que vê.

CAPÍTULO 2

Instalação

2.1 Instalar o Maprisma

O Maprisma é um único programa para Linux em computadores x86-64. Não precisa de Python e traz os seus próprios ícones e os seus add-ons assinados. Escolha uma destas formas:

- **Arch Linux e Omarchy (AUR):** instale o pacote `maprisma-bin` com o seu assistente AUR, por exemplo `yay -S maprisma-bin`. Coloca a aplicação em `/usr/lib/maprisma`, o comando `maprisma` no seu caminho e o Maprisma no lançador de aplicações.
- **AppImage, para outras distribuições:** descarregue `Maprisma-<versão>-x86_64.AppImage` de `github.com/maprisma/maprisma/releases`, torne-o executável (`chmod +x Maprisma-*.AppImage`) e execute-o. Usa o GTK 3 que quase todos os ambientes de trabalho Linux têm.

> **NOTA** O AppImage corre a partir de uma pasta temporária, por isso não instala o serviço em segundo plano de Definições › **Iniciar LLMs locais com os seus agentes**. Use o pacote AUR se os LLMs locais devem iniciar e parar com os seus agentes.

Programas opcionais: `xdg-terminal-exec` inicia agentes no seu próprio terminal, `xdg-utils` abre o manual e as páginas web, e `mise` instala e atualiza agentes.

O Maprisma guarda as definições em `~/.config/maprisma`; um `XDG_CONFIG_HOME` absoluto move essa pasta para `$XDG_CONFIG_HOME/maprisma`. No Omarchy usa sozinho a integração com o ambiente de trabalho: `maprisma --toggle` abre a janela como popup sob a barra, trá-la para a frente ou esconde-a.

2.2 Primeiro arranque

O pacote e o AppImage não precisam de mais configuração. Se usou o Maprisma com o seu nome anterior, Asterism, o primeiro arranque copia uma vez as pastas do Asterism; os originais ficam intactos.

A janela é uma janela de aplicação normal: pode ficar em mosaico, flutuar, ser movida para outra área de trabalho e alcançada com Alt+Tab. MAPRISMA Um Maprisma novo abre na sua **página de boas-vindas**: os agentes e LLMs locais instalados neste computador são cartões já assinalados, e **Ativar** coloca-os no mapa (ver Configurar o Maprisma). OMARCHY O botão da barra abre a janela, trá-la para a frente ou esconde-a se já estiver à frente. Fechar esconde a janela; o programa continua em segundo plano para voltar a abrir de imediato.

> **NOTA** Se houver um agente a correr dentro da janela, ao fechar pergunta-se primeiro: **Parar e fechar** (que também para o LLM local do agente e liberta a sua memória gráfica) ou **Manter a correr**.

Enquanto a aplicação vê o que há neste computador na primeira abertura, uma camada animada mostra o que encontra, tipo a tipo: agentes, LLMs locais, servidores MCP e skills, cada um com um indicador que passa a ✓ e um número, e quantos ainda não estão ligados. Desaparece sozinha após um momento; um clique fecha-a de imediato. A janela abre de imediato, com esta visão geral numa caixa ao centro; o mapa preenche-se por trás.

2.3 Remover a app

AUR: `sudo pacman -R maprisma-bin`. AppImage: apague o ficheiro.

Antes de remover, desligue Definições › **Iniciar LLMs locais com os seus agentes** para que o vigilante em segundo plano também seja removido. As alterações feitas com a aplicação ficam: definições dos agentes, servidores MCP, skills e LLMs locais instalados. Desinstale primeiro um LLM local (clique direito › **Desinstalar…**) se o quiser remover. Para apagar também o estado próprio da aplicação:

```
rm -rf ~/.config/maprisma ~/.cache/maprisma \
       ~/.local/share/maprisma ~/.local/state/maprisma
```

CAPÍTULO 3

A janela num relance

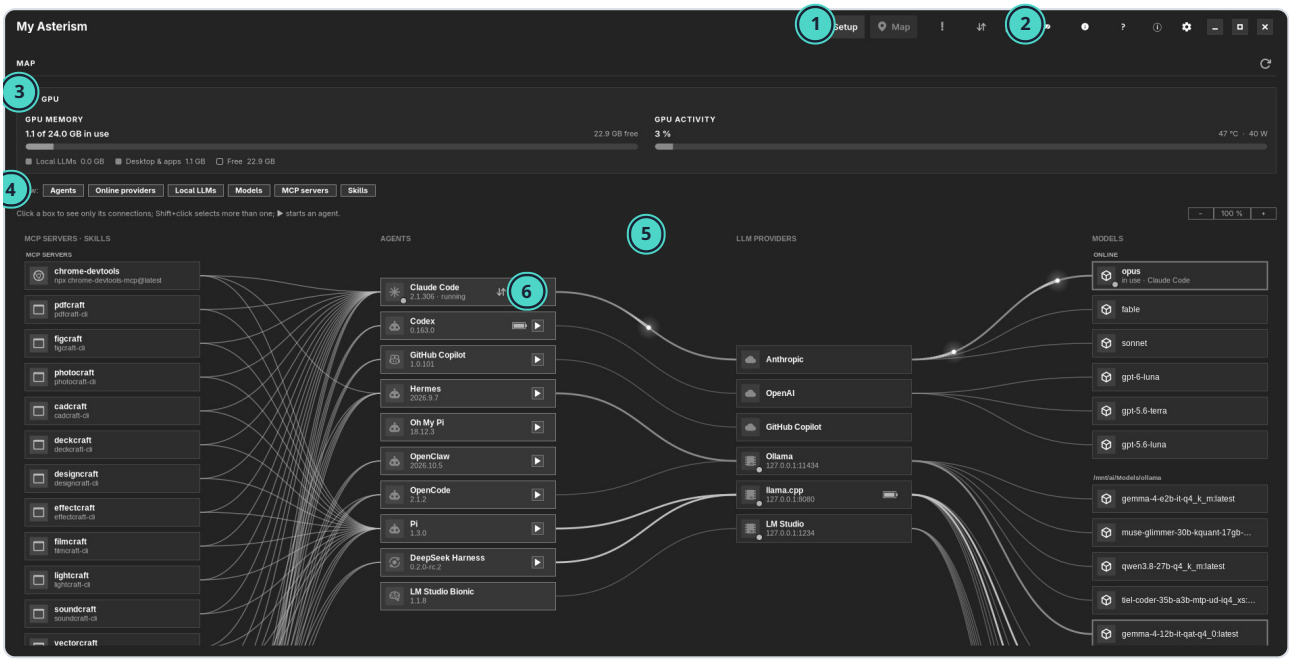


Figura 2 · A janela principal. Os números correspondem à lista abaixo.

1. **Configuração** e **Mapa** — as duas vistas, em todas as páginas; a que está à vista aparece realçada. Configuração tem cada agente, LLM local, servidor MCP, pasta de skills e modelo como um cartão: instalar, atualizar, iniciar sessão, remover (ver abaixo).
2. Os símbolos do cabeçalho: ↻ atualizar (`F5`: voltar a ver, como no arranque), **!** problemas por resolver e atualizações (vermelho: algo a corrigir; a cor de destaque: só atualizações e sugestões; não aparece quando não há nada), ⓘ a legenda (o que significa cada ponto, linha e marca do mapa), ⇅ o tráfego de rede dos agentes em execução (ver Tráfego de rede), as duas estrelinhas do **Smart Check** (só enquanto o Claude Code ou o Codex estiver instalado, ver Smart Check), os complementos MAPRISMA, o **i** preenchido para Informações (ver Ajuda), **?** Ajuda (`F1`) e a roda dentada das Definições (`Ctrl`+`,`). Mantenha o botão esquerdo do rato premido sobre um símbolo e arraste-o para o lado para o pôr noutro sítio: uma barra mostra onde fica, e a ordem é guardada.
3. **GPU** — memória da GPU e atividade da GPU: como a placa gráfica está a ser usada agora, num cartão que se recolhe (recolhido, a sua linha mantém os números).
4. Interruptores **Mostrar** — reduzem o mapa às partes em que está a trabalhar.
5. **O mapa** — servidores MCP e skills ← agentes → fornecedores de LLM → modelos, em toda a largura, sob o título **MAPA**.
6. ▶ na caixa de um agente inicia-o num terminal dentro da janela; enquanto corre lá, o botão traz de volta o seu terminal.

3.1 Configuração

O mapa mostra o que está configurado — agentes e LLMs locais instalados, os servidores MCP e pastas de skills que os agentes usam, e os modelos — e, esbatidos, os servidores MCP e pastas de skills encontrados neste computador que nenhum agente usa ainda, prontos a ligar com uma linha (ver Encontrado neste PC). Tudo o resto faz-se em **Configuração** (o botão no cabeçalho, ao lado de **Mapa**), um cartão por item, em secções que se recolhem com um clique na sua barra:

- **Como as coisas chegam ao seu mapa** — um cartão recolhido no topo; abra-o para ver cinco passos numerados: ligar (ou encontrar) o que usa, instalar o que falta, iniciar sessão num agente online, ligar desenhando, e o que aparece sozinho.
- **O que pode fazer com isto** — para que serve a configuração (ajuda no dia a dia, programação, imagens, voz, pesquisa e escrita), um mosaico cada com o que existe e o passo seguinte; recolhido, a sua linha diz quantos estão prontos.
- **Agentes de IA, LLMs locais** — todos os que a app conhece. Um cartão com contorno azul está instalado neste PC; um cinzento não, e tem **Instalar** onde os instalados têm **Desinstalar**. OMARCHY Um clique num cartão instalado abre os seus detalhes. MAPRISMA Um clique assinala ou desassinala o cartão (✓) e depois **Aplicar** ativa ou desativa o seu complemento. Desativar tira-o do mapa; nada é apagado.
- **Fornecedores online** MAPRISMA — o complemento dos fornecedores online; o seu menu tem **Entrar em …** para cada agente instalado que os usa.
- **Servidores MCP, Skills, Modelos** — um cartão para cada item instalado. Um clique abre os detalhes (num modelo, o seu menu). O cartão **+** no fim de uma secção acrescenta um: **Adicionar servidor MCP**, **Adicionar skills**, **Adicionar modelo**, **Obter complementos**.
- ⋯ ou um clique direito num cartão abre o mesmo menu que a sua caixa no mapa: atualizar, entrar, iniciar ou parar, detalhes, desinstalar ou remover, mover um modelo.
- Um ↑ **versão** amarelo num cartão significa que há uma atualização; o seu menu tem **Atualizar**.
- Um **!** vermelho num cartão significa um problema por resolver ali; passe o rato por cima para ver qual.
- Uma pequena bateria no cartão de um agente online mostra o que resta dos seus limites (uma sessão, uma semana): conta o mais apertado, amarela com um quarto ou menos, vermelha abaixo de um décimo; passe o rato por cima para ver cada limite e quando é reposto. No cartão de um LLM local em execução, a bateria é a memória da GPU ainda livre, com as mesmas cores; passe o rato por cima para ver quanta esse LLM local usa.
- O símbolo de atualizar (F5) na linha do cabeçalho volta a ver o que está instalado, configurado e com sessão iniciada, e repete as verificações do arranque (novas apps, add-ons, atualizações, problemas em aberto).

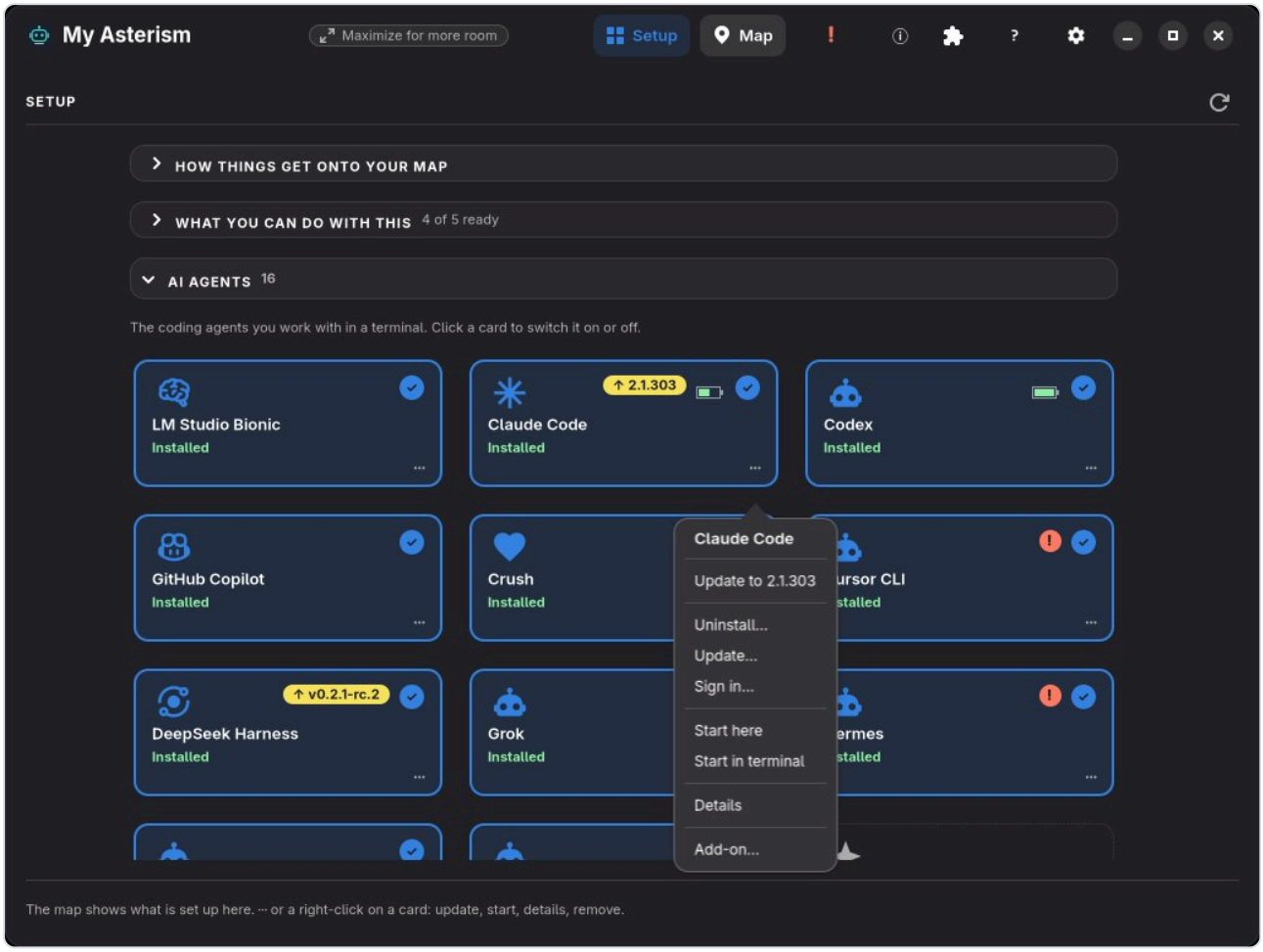


Figura 3 · Configuração: os agentes instalados têm moldura azul, os ligados um ✓, as atualizações um distintivo amarelo e os problemas um vermelho; o menu do Claude Code.

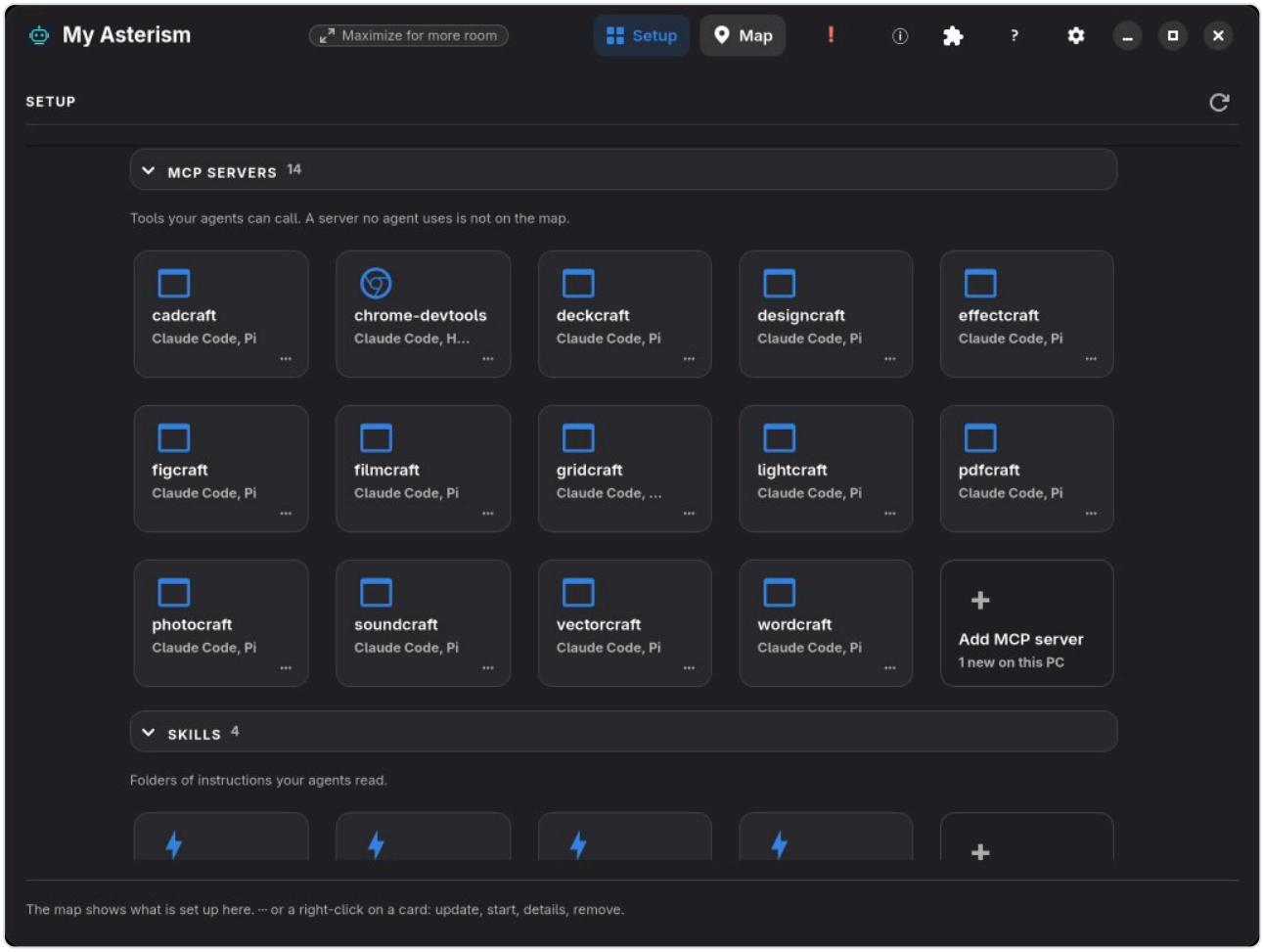


Figura 4 · Servidores MCP e skills, cada secção com um cartão para adicionar.

Não há nada para esconder: o que não quer no mapa, desativa-o ou remove-o aqui. Quando uma instalação ou um início de sessão corre num terminal dentro da janela, **Voltar** (ou **×** no seu separador) leva-o à página de onde partiu.

Servidores MCP locais e **Servidores MCP online** (os que se alcançam por um endereço web) são duas secções. **Adicionar pasta…** em **Servidores MCP locais** e em **Skills** adiciona uma pasta onde a aplicação também os procura — por exemplo um disco partilhado com aplicações extraídas que oferecem um servidor MCP, ou uma pasta com as suas próprias skills. As pastas adicionadas aparecem ao lado do botão; **×** numa deixa de procurar lá (nada é apagado). O que nelas for encontrado aparece esbatido no mapa até uma linha o ligar: a Configuração só tem cartões para o que está configurado.

3.2 Escolher o que o mapa mostra

O mapa mostra tudo o que está configurado. Os interruptores **Mostrar** por baixo do cartão GPU deixam de fora zonas inteiras — por exemplo todas as skills, ou os LLMs locais e os seus modelos — e ficam memorizados.

Numa janela estreita (meio ecrã ou menos) o mapa escolhe-as sozinho, e uma linha junto aos interruptores di-lo: agentes e fornecedores (quem trabalha através de quê). Um interruptor clicado ali mantém-se até a janela voltar ao tamanho completo e não fica memorizado; a janela em tamanho completo mantém a sua própria escolha. Mais áreas ligadas não apertam as caixas: o mapa desloca-se para o lado.

3.3 Trabalhar com o mapa

- **Passa o rato** por uma caixa para a realçar com tudo aquilo a que está ligada: os seus contornos ficam na cor de destaque e o resto do mapa esbate-se. Caso contrário, cada caixa tem o mesmo contorno cinzento fino.
- **Clica** numa caixa para a selecionar (✓); `Shift`+clique acrescenta mais. As caixas selecionadas e tudo aquilo a que estão ligadas mantêm o contorno na cor de destaque enquanto o ponteiro não estiver sobre outra caixa. `Esc` ou **Limpar seleção** limpa-a.
- O **pionés** no canto superior direito de uma caixa mantém as suas ligações realçadas enquanto olha para outro lado.
- **Clique direito** numa caixa para tudo o que o seu cartão em Configuração oferece: iniciar ou parar, detalhes, instalar, entrar, modelos, atualizar, remover e mais.
- ▶ na caixa de um agente inicia-o num terminal dentro da janela.
- Os agentes ficam sempre no centro da parte visível do mapa enquanto desloca.
- O caminho ativo de cada agente — agente → o fornecedor que usa → o seu modelo — é desenhado a negrito. Enquanto um agente trabalha, uma luz percorre essa linha.
- A **caixa de estatísticas de uso**: enquanto um agente trabalha, pequenos números são escritos ao longo da sua linha até ao fornecedor, na cor da linha: quão rápido chegou a sua última resposta (**t/s**, tokens por segundo), o tamanho do seu contexto (**1M**) e quanto está em uso (**%**, amarelo a partir de 75 %, vermelho acima de 90 %, como as baterias). Vêm dos próprios ficheiros de sessão do agente (só números; Claude Code, Codex e Pi). O que mostra e o tamanho do texto ficam nas Definições.
- Uma linha vermelha ponteada liga um agente a um fornecedor para o qual está configurado mas que agora não usa (nenhum modelo dele em uso).
- Quando os agentes usam modelos diferentes de um mesmo LLM local, as linhas para esses modelos são violeta: mantém-nos carregados ao mesmo tempo ou alterna-os (ver Modelos ao mesmo tempo).
- A caixa de um agente online tem a mesma bateria que o seu cartão em Configuração: o que resta dos seus limites; a caixa de um LLM local em execução, a memória da GPU ainda livre.
- Um agente pago por token — com sessão iniciada por uma chave de API, portanto sem limites de plano para uma bateria — mostra em vez disso quanto custam hoje as suas sessões ativas a preços de API, como um pequeno **"$3.40 hoje"** na sua caixa e no seu cartão: verde, amarelo a partir de 75 % do limite diário e vermelho acima (Configurações › **Custos**), atualizado a cada minuto. Os seus detalhes mostram todo o custo de API.
- Quando um programa que não está no mapa usa um LLM local, a linha desse LLM até ao seu modelo fica grossa e numa cor própria (roxo ou ciano, a que se destacar das outras cores do tema), e uma luz percorre-a. Ver Usado por outras apps.
- Quando um agente iniciou outro — o Claude Code a executar `pi -p …` ou `codex exec …` pela sua shell, por exemplo —, uma **seta** corre pela esquerda do cartão dos agentes, do agente que usa o outro até ao usado, com luz a percorrê-la; a sua dica nomeia ambos. Um agente pode usar vários outros ao mesmo tempo, cada um com a sua própria seta. Enquanto é usado, o agente usado tem também as suas linhas até ao seu fornecedor e daí até ao seu modelo na cor da seta, e a luz nelas corre nessa cor: vê de relance que fornecedor e que modelo trabalham para quem.

- O símbolo de atualizar (`F5`) na linha do cabeçalho volta a ver o que está instalado, configurado e com sessão iniciada, e repete as verificações do arranque (novas apps, add-ons, atualizações, problemas em aberto).
- Até a primeira linha ser desenhada, um cartão por cima do mapa mostra como em três passos; **Ver como** reproduz a animação da Ajuda. Depois da primeira ligação, ou com ×, desaparece para sempre.
- Zoom com **− 100 % +**, `Ctrl`+roda ou `Ctrl`+`+` / `-` / `0`.
- As caixas estão num cartão por categoria (agentes, fornecedores online, LLM locais, um cartão por pasta de modelos, servidores MCP, skills). Arrasta um cartão pelo cabeçalho para mudar a ordem na sua coluna, e uma caixa dentro do seu cartão para mudar a ordem.
- **Clica no cabeçalho de um cartão** para o recolher até ao cabeçalho (▸ à direita do cabeçalho) e outra vez para o abrir (▾). As linhas das caixas de um cartão recolhido terminam no seu cabeçalho, por isso continuas a ver o que lhe está ligado (as setas entre agentes ficam escondidas enquanto o cartão dos agentes está recolhido). Ao abrir um cartão, os cartões por baixo dele na sua coluna descem.
- O **pionés** à direita do zoom mantém a disposição do mapa — a ordem dos cartões e das caixas e que cartões estão recolhidos — depois de **Atualizar** (`F5`) e de um reinício. Sem fixar, como começa, o mapa volta à disposição predefinida em cada atualização e início; as tuas alterações duram até lá. **Layout padrão** repõe tudo de uma vez.
- As **caixas esbatidas** nos cartões de servidores MCP e skills estão neste computador, mas nenhum agente as usa. Trace uma linha de um agente até uma para a ligar (ver Encontrado neste PC). Os servidores MCP alcançados por um endereço web têm um cartão próprio, **Servidores MCP online**, ao lado de **Servidores MCP locais**; ambos se recolhem como qualquer outro cartão.

> **DICA** Numa janela estreita (menos de cerca de 1300 píxeis, mais ou menos meio ecrã) o mapa desloca-se para o lado.

3.4 Problemas por resolver

Quando há algo a corrigir, aparece um **!** vermelho entre os símbolos do cabeçalho; a sua dica diz quantos, por exemplo *2 problemas por resolver · 4 sugestões*. Um clique abre **Problemas por resolver**: uma linha cada — um agente sem sessão iniciada, um LLM local sem modelos, uma ligação que não funciona, pouco espaço livre para modelos — com um botão até onde se corrige (os detalhes do agente ou do LLM local, o mapa com essa linha realçada, Configuração, Arrumar). Os mesmos problemas aparecem como um **!** vermelho nos seus cartões em Configuração. As dicas e o que funciona estão nos próprios cartões e páginas de detalhes, e para que serve a configuração está em Configuração (**O que pode fazer com isto**). A verificação só lê; nunca altera, inicia ou envia nada, e não mostra nada enquanto o mapa está vazio.

3.5 Os problemas por resolver na pré-visualização Flutter

Na pré-visualização Flutter, um **!** no cabeçalho (vermelho quando há algo a corrigir) abre **Problemas por resolver**: cada coisa a corrigir — deste PC (espaço livre para modelos), dos agentes, dos LLM locais e modelos, ou das ferramentas — com um botão que abre onde se corrige. **O que pode fazer com isto** mostra os casos de uso (ajuda no dia a dia, vibe coding, imagens, voz, pesquisa e escrita)

como **Pronto**, **Quase** ou **Ainda não**, com o que existe para cada um e o passo seguinte. **Verificar de novo** volta a olhar. As verificações só leem o que a app já sabe, mais a placa gráfica, a memória e o espaço livre; nada é alterado. **Atualizações e sugestões** lista as versões mais recentes dos agentes e LLM locais instalados, cada uma verificada conforme foi instalada: as ferramentas desatualizadas do mise, o canal de lançamento próprio de um agente (stable ou latest no Claude Code) e os LLM locais que a própria app instalou. **Atualizar** corre `mise upgrade` ou o atualizador próprio do agente num terminal próprio, ou abre a pré-visualização de instalação com a nova versão; a web é consultada no máximo de seis em seis horas.

3.6 Atualizações e sugestões

Por baixo dos problemas, na mesma página, **Atualizações e sugestões** lista o que há a dizer: programas ou skills novos neste PC, o que está neste PC mas não no mapa, e atualizações para os agentes e LLMs locais instalados, cada um com o seu botão (**Adicionar servidor MCP…**, **Configuração**, **Atualizar**). Quando só há estes e nenhum problema, o **!** do cabeçalho aparece na cor de destaque. As atualizações são procuradas em segundo plano ao abrir a janela, no máximo de seis em seis horas. A procura em si não instala nada; uma atualização só começa quando clica em **Atualizar**.

Com Definições › **Verificações IA diárias** ligado, as atualizações de servidores MCP que um agente encontrou na web também aparecem aqui, com **Página da versão** e **Fazer com …**; quando a nova versão já está neste PC, a linha oferece antes **Usar…** (ver Atualizações encontradas).

3.7 Sugestões de complementos na pré-visualização Flutter

Na pré-visualização Flutter, a aplicação lê ao arrancar, em segundo plano, o catálogo assinado de complementos: da sua cache, e online só quando **Procurar complementos online** está ligado. Problemas por resolver mostra então em **Atualizações e sugestões**: os agentes e LLM locais deste PC cujo complemento está desligado ou ainda não instalado (**Obter complementos** ou **Complementos** abre os complementos para ligar ou obter), e os complementos instalados com uma versão mais recente no catálogo. **Atualizar tudo** mostra-os com as suas versões e, depois de confirmar, atualiza-os um após o outro, cada um verificado contra o catálogo; uma mensagem diz o que foi atualizado e o que não foi.

3.8 Cartões inteligentes no mapa

Na pré-visualização Flutter, quando pelo menos um agente ajudante protegido está pronto, aparece uma barra recolhida **Perguntar à IA…** no topo de cada coluna do mapa. Um clique abre um cartão sobre essa coluna sem mover o mapa. Antes de enviar seja o que for, o cartão mostra qual ajudante responde (com um menu quando há mais do que um pronto) e se corre localmente (**Local, sem tokens**) ou **Usa plano/tokens**; só clicar em **Perguntar** envia o pedido. O ajudante faz uma pergunta de esclarecimento, propõe mover o pedido para a coluna certa ou sugere candidatos verificados a partir de resultados de pesquisa reais. Na coluna **Modelos**, o cartão mostra a VRAM, a RAM e o espaço livre deste PC e assinala cada candidato como **Cabe na VRAM**, **Usa offload para RAM** ou **Demasiado grande para este PC**. **Rever** abre a pré-visualização de transferência antes de transferir ou alterar qualquer coisa e, quando o modelo está pronto, pode ligá-lo opcionalmente a um agente através da pré-visualização de ligação com **Anular**.

CAPÍTULO 4

Agentes

4.1 Detalhes de um agente

Cada agente de programação tem um cartão em Configuração, os instalados com a sua versão. Clique no cartão de um agente instalado (MAPRISMA ou ··· › **Detalhes**), ou clique direito na sua caixa do mapa › **Detalhes**. No topo, uma linha diz se o agente está pronto (e se não, porquê) e se está a correr agora ou quando foi usado pela última vez. Por baixo, cada parte é um cartão que se recolhe: um clique no título abre-o ou fecha-o. Começam recolhidos, cada um com um resumo de uma linha e distintivos; uma parte que abra continua aberta nas páginas de detalhes seguintes.

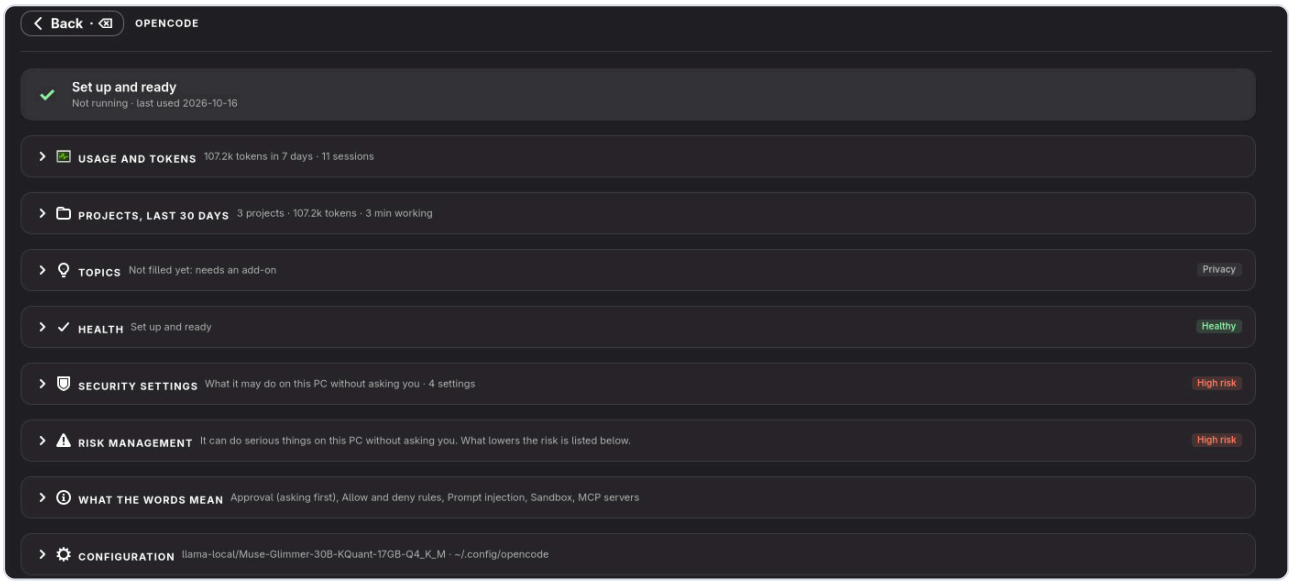


Figura 5 · Os detalhes de um agente: cada parte recolhida, com o seu resumo e distintivos.

Cartão	O que mostra
Uso e tokens	O que o agente regista: tokens e sessões de hoje e dos últimos sete dias, o plano, um gráfico de tokens por dia, limites com a hora de reposição e tokens por modelo.
Custo de API / **Valor de API**	Quanto custam os seus tokens aos preços de API dos fornecedores: hoje, os últimos 7 e 30 dias e, por modelo, a divisão em entrada, saída, leitura de cache e escrita de cache (tokens e montante). **Custo de API** para um agente pago por token (com a sua parte do limite diário), **Valor de API** para um com plano (o plano cobre-o). Veja Custo de API.
Projetos, últimos 30 dias	Por pasta de projeto: tempo de trabalho, sessões, tokens, execuções de subagentes, linhas alteradas e o valor de API.
Tópicos (Privacidade)	Sobre o que eram as sessões. Vai além dos números, por isso está marcado com **Privacidade** e só um complemento o preenche (ver abaixo); o complemento dá nome ao cartão e o seu distintivo diz quanto lê.

Cartão	O que mostra
Estado	Cada verificação da configuração com um ✓, ou o que fazer a seguir: entrar, instalar ou iniciar o seu LLM local, descarregar um modelo, adicionar servidores MCP ou skills.
Definições de segurança, **Gestão de riscos**, **O que significam as palavras**	O que o agente pode fazer sem lhe perguntar e o que reduz o risco (ver Verificação de saúde e segurança).
Configuração	As definições do agente, servidores MCP, skills e a sua pasta de definições.

Os projetos são mostrados para todos os agentes cujas sessões a app lê — o Claude Code, o Codex, o Pi, o Oh My Pi, o OpenCode, o Hermes, o DeepSeek Harness, o Goose, o Gemini CLI e o OpenClaw: tempo de trabalho, sessões, tokens, execuções de subagentes, linhas alteradas e quanto custariam os tokens a preços de API. Os números vêm dos próprios ficheiros de sessão ou da base de dados de cada agente; nenhum texto é lido.

Custo de API. Os montantes são uma estimativa: as contagens de tokens que o agente guarda (entrada, saída, leitura e escrita de cache, por modelo) aos preços de API publicados pelos fornecedores — Anthropic, OpenAI, DeepSeek, xAI e Google, com a data em que foram tirados. Uma sessão conta no dia da sua última resposta; os modelos locais não custam nada. O Crush não guarda contagens de tokens que se possam somar, por isso o seu cartão mostra o custo que o próprio Crush indica. Para um agente que não guarda contagens de tokens no PC (Cursor CLI, Antigravity, Muse) o cartão diz que o custo é desconhecido; a página da conta do seu fornecedor tem-no. Os montantes aparecem em dólares americanos, ou em euros à taxa de referência diária do Banco Central Europeu (Configurações › **Custos**).

MAPRISMA Com o complemento **Insights** (ver Complementos) o cartão **Tópicos** mostra primeiro que temas usaram mais tokens nos últimos 30 dias (**Mais tokens**: sessões com o mesmo título são um só tema, cada um com a sua parte de todos os tokens), depois os títulos que o agente deu às suas sessões, por projeto (**Por projeto**), e oferece **Resumir com o Claude…** (quando o Claude Code está instalado). Os seus tokens vêm dos próprios ficheiros de sessão do agente sem as leituras de cache, por isso podem ser muito mais baixos do que em **Uso e tokens**, que pode contar as leituras de cache. Antes de qualquer envio, vê exatamente o que vai ser enviado: os títulos e os números, nunca as conversas.

4.2 Iniciar e parar agentes

São guardadas as últimas 50 pastas de projeto por agente.

O submenu **Iniciar** de cada agente tem três secções separadas por linhas: **Iniciar** com a pasta atual, **Escolher pasta…** e a lista pendente com pesquisa **Projetos recentes**. A lista desloca-se e mostra todas as pastas usadas para iniciar este agente a partir do mapa, das mais recentes para as mais antigas. Escolha um projeto anterior e depois Iniciar. O seletor de pastas inicia uma sessão na pasta escolhida. Não é obrigatório fixar uma pasta: sem uma, Iniciar usa Definições › **Os agentes iniciam em** ou a pasta pessoal. A página de detalhes tem um cartão deslocável **Projetos recentes** com todas as pastas registadas. Clique numa linha para abrir Iniciar com essa pasta selecionada. O alfinete define a próxima pasta predefinida deste agente; só pode fixar uma pasta por agente. Desafixar repõe a

predefinição geral. O caixote do lixo remove a entrada e o alfinete do histórico deste agente, nunca a pasta ou os ficheiros. O mesmo submenu, histórico e alfinetes aplicam-se a **Iniciar aqui** e **Iniciar no terminal**. **Iniciar novamente** volta a abrir o submenu. As sessões ativas mantêm a pasta; escolhas canceladas e arranques falhados não acrescentam histórico.

- ▶ na caixa do agente ou **Iniciar aqui** no seu menu inicia o agente num terminal dentro da janela, em vez do mapa. Precisa do VTE (`vte4`).
- **Iniciar no terminal** abre-o na sua própria janela de terminal.
- Enquanto corre dentro da janela: **Mostrar terminal** e **Parar**. Parar termina o agente e tudo o que ele iniciou, como os seus servidores MCP.
- **Terminais…** lista onde mais o agente corre: *Aqui, nesta janela*, cada *Janela de terminal* (um clique traz-a para a frente) e cada instância *Em segundo plano* — uma sem janela, como `claude -p …` executado por um script ou por outro agente. Um clique numa instância em segundo plano abre um separador de terminal que mostra o que ela escreve, à medida que escreve: perguntas, respostas e as ferramentas que usa (do seu ficheiro de sessão, ou da sua saída quando esta vai para um ficheiro). O separador termina quando a instância termina. Este é o único sítio onde a app mostra texto de conversas, e só para a instância em que clica.
- **Parar…** — também o pequeno botão de parar à esquerda de ▶ na caixa do agente — para uma das suas instâncias fora desta janela. Escolha-a da lista, que diz onde e quando cada uma foi iniciada. É-lhe pedido que termine (SIGTERM) e, se ainda lá estiver passados 3 segundos, é terminada. Só esse mesmo processo é parado: se entretanto terminou e o seu número de processo pertence agora a outro programa, nada acontece.

Quando há agentes com terminais dentro da janela, uma fila de separadores por cima do mapa mostra um separador por terminal, com um ponto para *a trabalhar*, *a correr* ou *terminado*. Um clique num separador mostra esse terminal; **×** fecha um terminado.

Um agente aberto em qualquer lado do computador recebe um ponto verde e *a correr*, e a caixa do modelo com que corre diz *em uso · <agente>*.

4.3 Tráfego de rede

A caixa e o cartão de um agente em execução mostram o que recebe e envia (↓ entrada, ↑ saída). Clique nessa linha, ou em ⇅ no cabeçalho para todos os agentes em execução, para abrir a sua página de rede: **Agora**, **Os últimos 5 minutos** como gráfico (entrada a verde, saída a azul), **Para onde vai** — API online, LLM locais, este PC e a sua rede local, cada um com a sua parte — e **Programas**: o próprio agente e os programas que ele iniciou, como os seus servidores MCP. É contado a partir das ligações TCP do próprio agente enquanto a app está aberta (sem direitos de administrador; o UDP não é contado) e fica neste PC.

4.4 Subagentes

O Claude Code e o OpenCode podem passar partes de uma tarefa a **subagentes**. Enquanto um dos subagentes de um agente trabalha, a sua caixa no mapa mostra uma pequena marca em árvore ao lado de ⇅; se só tiver terminados na lista, a marca fica esmaecida. Clique nela para abrir a página **Subagentes** do agente: um mapa próprio, disposto como o grande, com o cartão do agente à esquerda e um cartão **Subagentes** à direita. Cada caixa mostra a tarefa do subagente, o seu tipo, a

pasta onde a sua sessão trabalha e **a trabalhar** ou **concluído há … minutos**. A luz corre nas linhas para os que trabalham; os terminados ficam esmaecidos. Passe o ponteiro sobre uma caixa para ler toda a tarefa.

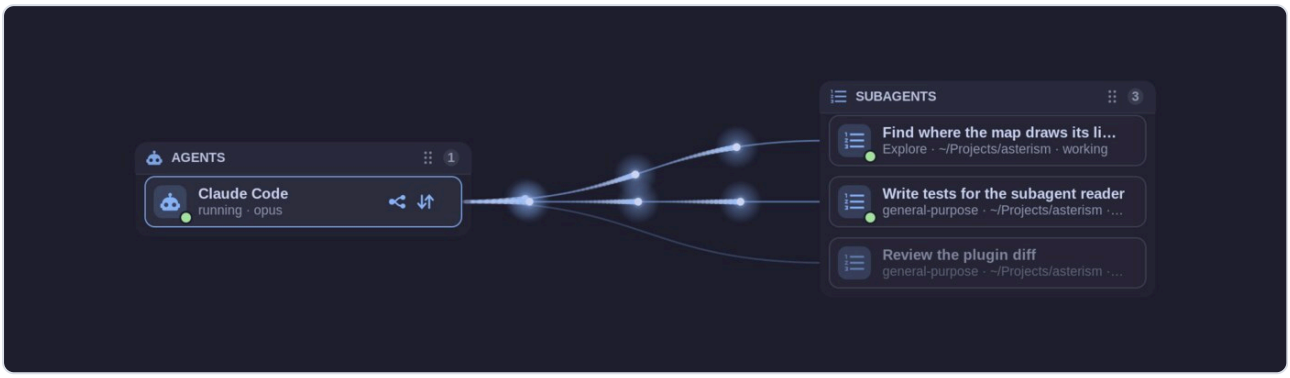


Figura 6 · Os subagentes de um agente: dois a trabalhar, um concluído.

Quanto tempo os subagentes terminados ficam na lista (30 minutos) e quantos no máximo (10) define-se em Configurações › **Subagentes**; os que trabalham são sempre mostrados. A página só lê o tipo de cada subagente e a descrição de uma linha da sua tarefa, a partir dos ficheiros do próprio agente (as pastas de sessão do Claude Code, a base de dados do OpenCode), nunca o que os subagentes dizem.

4.5 Agentes que trabalham juntos na pré-visualização Flutter

Na pré-visualização Flutter, um agente que iniciou outro e o usa — o Claude Code a correr `pi -p …` pela sua shell, por exemplo — fica ligado a ele no mapa por uma linha curvada à direita das caixas dos agentes. A luz corre nela enquanto um dos dois trabalha. Deteta-se só pela árvore de processos (os processos dos agentes e os seus pais), e a linha é mostrada, nunca editada. Os subagentes do Claude Code, Codex, GitHub Copilot, OpenCode e Goose são lidos como na app.

4.6 Custos na pré-visualização Flutter

Os detalhes de um agente mostram na pré-visualização Flutter quanto custam os seus tokens aos preços de API dos fornecedores: hoje, nos últimos 7 e 30 dias, e por modelo dividido em entrada, saída, leitura de cache e escrita de cache. **Custo de API** para um agente pago por token (com a parte do limite diário), **Valor em API** para um com plano (que o cobre). É uma estimativa a partir dos números dos seus próprios ficheiros de sessão; nenhum texto deles é lido. Configurações › **Custos** define a **Moeda** (euros à taxa de referência diária do Banco Central Europeu, obtida uma vez por dia), o **Limite diário** e os níveis **Amarelo a partir de** e **Vermelho a partir de** (vermelho sempre acima do amarelo). Os ficheiros de sessão do Claude Code são lidos a partir de onde a última leitura parou, por isso sessões muito longas contam inteiras; a primeira leitura num PC com muitíssimas pode precisar de duas para ficar completa.

No mapa, um agente com limites do plano mostra na sua caixa uma pequena bateria com o que resta do limite mais apertado, e um pago por token mostra lá em vez disso o custo de hoje; verde, amarelo ou vermelho segundo os níveis de Configurações › Custos, os detalhes na sua dica. Ambos são vistos de novo a cada minuto. Com **Avisar no vermelho** ligado, um custo no vermelho recebe uma notificação no ambiente de trabalho por dia.

4.7 Detalhes na pré-visualização Flutter

Os detalhes de um agente mostram também **Projetos, últimos 30 dias**: por pasta de projeto o tempo de trabalho, as sessões, os tokens, as linhas alteradas e o valor em API, as execuções de subagentes por tipo e a hora mais ativa, a partir dos números dos seus ficheiros de sessão. O cartão de um LLM local diz se está a correr, quanta memória de GPU ocupa e que modelos estão carregados, mostra os seus modelos, agentes e endereço, **Uso e tokens** que os agentes enviaram através dele (associados pelo nome do modelo) e, em **Gestão de riscos**, quem o pode alcançar. No mapa, um LLM local a correr mostra a memória de GPU ainda livre como uma bateria. Um clique na caixa de um servidor MCP ou de um conjunto de skills abre os seus detalhes: que agentes o têm e como cada um o inicia, chama ou lê (os valores de chaves, tokens e parâmetros de consulta nunca são mostrados), quem ainda não o tem, e as ferramentas do servidor como pedidas da última vez ou a descrição de cada skill; **Mapa do sistema** abre o mapa do sistema como antes.

4.8 Separadores de terminal, caixa de uso e modelo inicial na pré-visualização Flutter

Enquanto houver agentes em terminais na janela, acima do mapa há um separador para cada um, com um ponto para **a trabalhar**, **em execução** ou **terminado**; um clique mostra o terminal e o visível fica marcado (Configurações: **Separadores de terminal acima do mapa**). A caixa de um agente a trabalhar no mapa mostra a **Caixa de estatísticas de uso**: quão rápida foi a sua última resposta (tokens por segundo), o tamanho do seu contexto e quanto está em uso (amarelo a partir de 75 %, vermelho acima de 90 %), lidos dos seus próprios ficheiros de sessão, só números; as Configurações dizem o que aparece e em que tamanho. Em **O seu primeiro agente**, uma rota com um LLM local sugere o modelo inicial que cabe na sua placa gráfica (**Um modelo**), e **Baixar** inicia o seu download como a página de modelos.

4.9 Configurar…

Clique com o botão direito num agente ou num LLM local › **Configurar…** abre a sua própria configuração num separador de terminal da janela, para o que a app não define por si — por exemplo `hermes setup` ou o menu do Ollama; o LM Studio abre a sua janela. Um programa sem comando de configuração mostra o seu `--help`. Quando termina, o mapa volta a ver e mostra o que mudou, e a um LLM local em execução é proposto um reinício.

4.10 Entrar

Entrar… no menu de um agente (em Configuração ou no mapa) mostra o que o agente precisa e as suas próprias formas de iniciar sessão:

- **Uma subscrição ou uma conta** — o início de sessão do próprio agente corre num terminal dentro da janela (por exemplo `claude auth login` ou `codex login`). A sua palavra-passe ou o início de sessão no navegador ficam entre si e o agente; a app nunca os vê. Quando o agente o pede, o terminal diz o que escrever (Pi e Copilot: `/login`).
- **Uma chave de API** — para os agentes que leem uma: a chave vai para um campo de palavra-passe e daí diretamente para o próprio comando do agente (o Codex guarda-a ele mesmo) ou para o

ficheiro que o agente lê (Gemini: `~/.gemini/.env`, legível só por si). Nunca vai parar a uma linha de comandos, a um registo ou às definições da app, e a app não a guarda. A faturação por API do Claude Code é o seu próprio início de sessão na Anthropic Console, por isso nenhuma chave passa pela app.

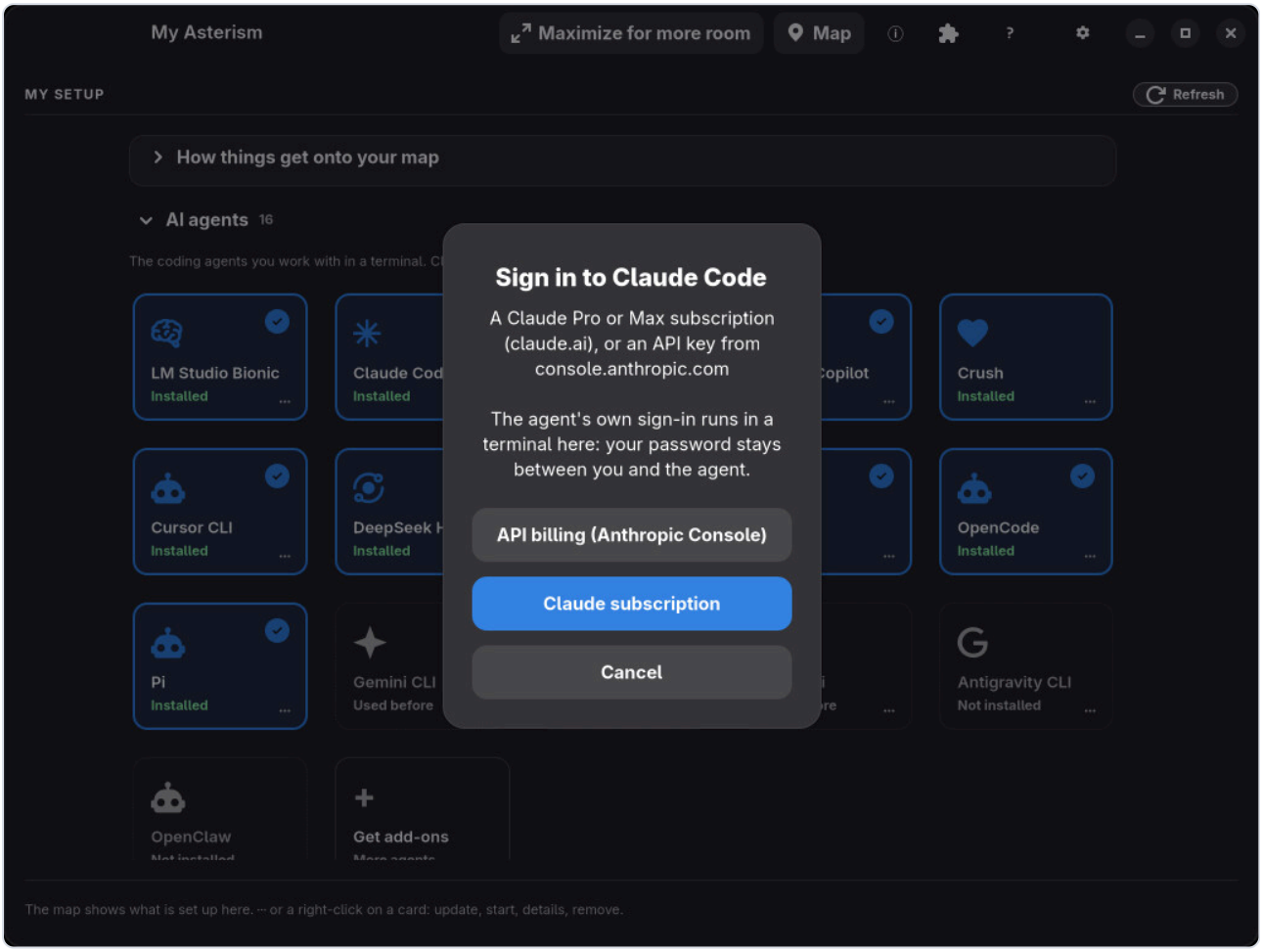


Figura 7 · Entrar no Claude Code: uma subscrição ou faturação por API, ambas com o início de sessão do próprio Claude Code.

Depois, o mapa mostra o fornecedor online do agente e os seus modelos.

4.11 Chaves de API na pré-visualização Flutter

Na pré-visualização Flutter, um agente que lê uma chave de API mostra **Chave de API…** nos seus detalhes. O cartão diz para onde vai a chave antes de a escreveres: para o comando próprio do agente (Codex: `codex login --with-api-key`) ou para o ficheiro que o agente lê (Gemini: `~/.gemini/.env`, escrito como um ficheiro novo que só tu podes ler; as outras linhas ficam). O campo esconde o que escreves e **Guardar** só fica disponível para algo com forma de chave. A chave vai para um processo de trabalho separado pela sua entrada padrão; nunca entra numa linha de comandos, num registo, numa cópia de segurança nem nas definições da aplicação, e a aplicação não a guarda. Quando o comando próprio do agente responde, é mostrada a sua última linha, sem a chave.

4.12 Atualizar agentes

Quando há uma versão mais recente de um agente instalado, o seu cartão em Configuração recebe um ↑ **versão** amarelo, e **Atualizações e sugestões** (o **!** do cabeçalho) menciona-a. **Atualizar** executa a atualização da mesma forma que o agente foi instalado — por exemplo `mise upgrade <ferramenta>` ou `claude update` — na sua própria janela de terminal, para que a possa acompanhar.

4.13 Atualizar um agente em execução na pré-visualização Flutter

Quando clica em **Atualizar** para um agente que está a ser executado neste momento — num separador de terminal aqui ou num terminal próprio —, a pré-visualização Flutter avisa primeiro: uma atualização substitui o programa do agente enquanto trabalha, por isso as suas sessões podem parar ou comportar-se de forma estranha, e alguns atualizadores recusam. **Cancelar** deixa tudo como está; deixe o agente terminar e atualize mais tarde. **Atualizar mesmo assim** inicia a atualização num terminal como de costume.

4.14 Agentes que a app pode instalar

Quatro agentes podem ser instalados, atualizados e desinstalados a partir do mapa (clique direito no agente › **Instalar…**):

- **Goose** (`goose` , da Agentic AI Foundation, antes Block), a sua versão do GitHub através do mise. Funciona com muitos fornecedores e com LLMs locais; **Configurar…** executa `goose configure` .
- **DeepSeek Harness** (`dsh` , pré-visualização para programadores), a partir do npm. Pode usar LLMs locais.
- **Antigravity CLI** (`agy` , Google), a partir do manifesto de versões da Google; a transferência é verificada com a sua soma SHA-512.
- **LM Studio Bionic** não pode ser instalado daqui (sem soma de verificação publicada); obtenha-o em lmstudio.ai. Usa sempre o motor e os modelos do LM Studio.

4.15 Agentes novos

Quando aparece um agente que a app ainda não conhece, ela pede uma vez a um agente instalado (Claude Code, senão Codex) que descubra na web onde esse agente guarda as suas skills e servidores MCP. A resposta é verificada antes de ser usada, e nada dela é alguma vez executado. Definições › **Pesquisar agentes novos** desliga isto.

CAPÍTULO 5

LLMs locais

Um motor de LLM local corre modelos de linguagem na sua própria placa gráfica, por isso os pedidos nunca saem do computador. A app instala-os, inicia-os e para-os por si.

5.1 Instalar um LLM local

Clique direito num LLM local cinzento (não instalado) no mapa › **Instalar…**. Escolhe a pasta do programa, a pasta dos modelos (é sugerida a que os seus outros LLMs locais usam) e se arranca ao iniciar sessão. É instalado só para o seu utilizador, com um serviço de utilizador, por isso não é preciso palavra-passe. Cada transferência tem de corresponder à soma SHA-256 publicada pela sua versão antes de ser desempacotada ou executada.

Motor	Notas
Ollama	A versão oficial (com ROCm para placas AMD).
llama.cpp	A compilação que escolher (Vulkan, ROCm, CUDA, CPU), como encaminhador de modelos: qualquer GGUF na pasta dos modelos, um carregado de cada vez.
LM Studio	Não é instalado daqui (sem soma de verificação publicada); obtenha-o em lmstudio.ai. Uma vez instalado, é mostrado e gerido.
vLLM	Para placas NVIDIA, no seu próprio ambiente Python.
Jan	O seu AppImage, desempacotado, com o motor do Jan como serviço em segundo plano para os agentes.
KoboldCpp	A compilação oficial num só ficheiro, como encaminhador de modelos.

Instalado assim, um LLM local oferece também **Atualizar…** (a versão anterior fica como recurso), **Pasta dos modelos…** e **Desinstalar…**, que remove o programa, o comando, o serviço e as definições, mas nunca os modelos. Os LLMs locais que instalou por si são mostrados, mas nunca alterados.

5.2 Instalar na pré-visualização Flutter

Na pré-visualização Flutter, um LLM local ainda não instalado mostra **Instalar** em Fornecedores e modelos, e o guia do primeiro agente oferece **Instalar mise**, **Instalar Node.js** e **Instalar** para o LLM local de que o seu caminho precisa. O Maprisma procura primeiro a versão mais recente e mostra-a num cartão: os seus ficheiros e o tamanho, a pasta do programa (`~/.local/share/ai-agents-map/<nome>` , uma pasta por versão), a pasta de modelos dentro dela e a tua placa gráfica. Para llama.cpp e KoboldCpp, escolhe aí a variante (Vulkan, ROCm, CUDA ou CPU); outra variante é procurada de novo antes de instalar seja o que for. Um LLM local com um serviço de utilizador oferece também **Iniciar já** e **Iniciar ao entrar**.

Instalar descarrega os ficheiros, verifica cada um com a soma de verificação indicada pela versão e descompacta-os; nada da transferência é executado durante a instalação (o AppImage do Jan descompacta-se a si próprio numa sandbox isolada). Um programa já instalado de outra forma, ou um ficheiro de definições ou um serviço que já existe, nunca é substituído: o cartão recusa-o. Ollama, llama.cpp, KoboldCpp, Jan e mise instalam-se assim; LM Studio e vLLM não, porque não há para eles uma versão com soma de verificação.

Cada passo iniciado a partir do guia do primeiro agente — instalar, entrar, a pergunta de teste, Fornecedores e modelos, Ligações — leva-te de volta ao guia com **Voltar**, atualizado, para que o passo acabado de fazer apareça como feito.

5.3 Iniciar e parar com os agentes

Ligue uma vez um agente a um LLM local e depois basta iniciar o agente — mesmo com a app fechada ou depois de reiniciar. O seu LLM local arranca com ele, e quando termina o último agente que o usa, o LLM para cerca de cinco segundos depois, ficando de novo livre a sua memória gráfica.

Disso trata um pequeno serviço em segundo plano, `ai-agents-map-providers.service`. É configurado com a primeira ligação a um LLM local e removido quando desaparece a última ou quando desativa Definições › **Iniciar LLMs locais com os seus agentes**.

> **NOTA** O KoboldCpp demora alguns segundos a arrancar, mais do que alguns agentes esperam. Por isso arranca ao iniciar sessão, e quando termina o último agente só o seu modelo é descarregado da memória.

5.4 Detalhes de um LLM local

Clique no cartão de um LLM local em Configuração, ou clique direito na sua caixa do mapa › **Detalhes**. Por baixo dos mosaicos (modelos, agentes, endereço) vêm os mesmos cartões recolhíveis dos agentes: **Uso e tokens** — o que os seus agentes enviaram através dele nos últimos dias, por agente e modelo, a partir dos próprios números de sessão dos agentes e associado pelo nome do modelo —, depois Estado, Gestão de riscos, O que significam as palavras e Configuração.

5.5 Desempenho

Um LLM local cujas definições a app conhece (llama.cpp, Ollama instalado pela app) tem um cartão **Desempenho** na sua página de detalhes:

- **Variante**, **Pilhas de GPU** e **Memória de vídeo**: com que corre — ROCm, Vulkan, CUDA ou o CPU (no llama.cpp lido da sua própria lista de dispositivos, também numa variante compilada por si), quais destas pilhas este PC tem e a memória da placa.
- **Contexto por modelo**: o tamanho do ficheiro de cada modelo, o contexto definido agora, o contexto que a placa comporta para ele — o seu ficheiro mais a cache do contexto, lida do cabeçalho do modelo — e aquele para que foi treinado. **!** marca um contexto maior do que a placa comporta (o modelo não carregaria, ou transbordaria para a RAM), ↑ um que a placa poderia aumentar; **Definir para …** altera-o.

- **Perguntar a …**: o ajudante local (ou o agente online) examina estes dados — só nomes e números — e sugere alterações: definições como flash attention, o tipo de cache do contexto, as camadas na GPU, os pedidos em paralelo e os tamanhos de lote; contextos; outra variante quando ajudaria claramente. A app só mantém as sugestões que consegue garantir (uma definição conhecida com um valor permitido, um contexto que a placa comporta) e mostra cada uma com o motivo. **Aplicar** escreve-a com cópia de segurança; a mensagem oferece **Desfazer**, ou **Reiniciar agora** quando só conta depois de um reinício. Outra variante é apenas sugerida: instale-a a partir do menu do LLM local.

No Ollama, as definições valem para todos os modelos de uma vez (o ficheiro de ambiente do seu servidor).

5.6 O desempenho na pré-visualização Flutter

Na pré-visualização Flutter, um LLM local com definições que a app pode afinar (llama.cpp, Ollama) tem **Desempenho** em **Fornecedores e modelos**. Mostra a compilação com que corre, as pilhas de GPU deste PC e a memória de vídeo, depois **Contexto por modelo**: o tamanho de cada modelo, o seu contexto atual, o que a placa aguenta para ele e para que foi treinado. Um modelo com mais contexto do que a placa aguenta fica marcado, tal como um para o qual a placa ainda tem espaço; no llama.cpp, **Definir para …** define-o. **Definições para todos os modelos** lista as definições do motor com o que cada uma faz; uma com uma lista de valores pode ser alterada aí. Cada alteração é mostrada primeiro e escrita com uma cópia de segurança, pelo que **Desfazer** a repõe; conta depois de reiniciar o LLM local. Em **Dicas de um agente**, **Perguntar a …** faz o agente das verificações de IA olhar para estes factos (só nomes e números, só pela web): sugere definições, contextos, outra compilação ou notas, cada um com o porquê. A app só guarda o que pode garantir — um valor permitido, um contexto que a placa aguenta, uma compilação que o LLM local tem — e **Aplicar** passa pela mesma pré-visualização. Os tokens são do agente; a execução aparece no Smart Check.

5.7 Memória da GPU e libertar memória da GPU

No cartão **GPU** por cima do mapa, **Memória da GPU** mostra quanta está em uso, repartida por LLMs locais, ambiente de trabalho e aplicações, e livre; **Atividade da GPU** mostra a carga, a temperatura e a potência. A caixa de cada LLM local mostra quanta memória ocupa, e os modelos carregados recebem um ponto verde.

Clique direito num LLM local para **Iniciar**, **Parar**, arrancar ao iniciar sessão e **Libertar memória da GPU**: os seus modelos são descarregados da memória, continua a correr e volta a carregar um quando lhe pedem.

Quando o último agente que usava um LLM local fecha e o LLM ainda tem os seus modelos carregados, uma mensagem pergunta se a memória deve ser devolvida: **Libertar** descarrega-os, **Manter** deixa-os carregados.

Um LLM local que ocupa memória da GPU recebe um botão vermelho de parar na sua caixa. Ele pergunta antes: **Libertar memória da GPU** descarrega seus modelos; **Parar** o encerra. No LM Studio o botão é **Parar servidor**: só o servidor para, e a app continua aberta com os modelos carregados. Um modelo que um LLM local carregou, mas que nenhum agente usa, fica ligado a ele por uma linha clara contínua.

O LM Studio pode estar aberto sem o seu servidor. A sua caixa mostra então *servidor desligado* com um ponto amarelo, os agentes não o alcançam e o seu menu oferece **Iniciar servidor** (`lms server start`).

5.8 Modelos ao mesmo tempo

O llama.cpp, o LM Studio e o Ollama podem manter mais de um modelo carregado. Quando liga agentes a modelos diferentes de um mesmo LLM local, a app ajusta quantos ele mantém carregados ao mesmo tempo ao número que os seus agentes usam — com a mesma pré-visualização, cópia de segurança e Anular da ligação — e propõe reiniciá-lo se estiver em execução, porque ele lê esse número ao arrancar. Onde esse número não cabe à app (uma instalação do sistema ou o seu próprio comando de arranque), uma nota diz onde o definir.

Quando os modelos juntos não cabem na memória da GPU, um aviso di-lo: o LLM local alterna-os então, o que é lento, ou parte de um modelo corre no processador. Iniciar um agente verifica o mesmo: se carregar o seu modelo precisar de mais memória da GPU do que a livre, é-lhe perguntado antes, com o que ocupa a memória agora — **Libertar memória da GPU e depois iniciar**, **Iniciar mesmo assim** ou **Cancelar**.

5.9 Modelos em simultâneo na pré-visualização Flutter

Na pré-visualização Flutter, depois de aplicada uma alteração do LLM local ou do modelo de um agente, a app vê quantos modelos diferentes os agentes de cada LLM local usam agora. Onde pode definir esse número — os ficheiros de definições do llama.cpp e do Ollama das instalações da própria app, a definição «um ou vários» do LM Studio —, uma segunda **Pré-visualização** mostra a alteração, com o que significa e eventuais avisos; **Aplicar** escreve-a com uma cópia de segurança, e **Desfazer** volta atrás. Um número fixado no teu próprio script de arranque fica como está. Quando os modelos são mais do que os mantidos carregados, ou demasiado grandes juntos para a placa gráfica, os Problemas por resolver também o dizem.

5.10 Usado por outras apps

Os LLMs locais não são usados só por agentes. Um plugin como o Video Trans, uma app de chat ou o seu próprio script também podem enviar-lhes pedidos. A app repara nisso e mostra-o:



Figura 8 · Um programa que não está no mapa (aqui `curl`) a gerar com o llama.cpp: a linha e a caixa ciano (tema Tokyo Night). O próprio caminho do Claude Code até ao opus fica na cor de destaque.

- **No mapa**, a linha do LLM local até ao modelo com que gera fica grossa e na sua própria cor, uma luz percorre-a, e a caixa do modelo recebe um ponto nessa cor e diz quem o usa, por exemplo *em uso · Video Trans*.
- **O menu de clique direito do LLM local** e a sua página ⓘ **Detalhes** dizem *Em uso pelo Video Trans · agora* ou há quantos segundos foi usado pela última vez.
- A legenda ⓘ mostra ambas as marcas.

A app vê quais dos seus próprios programas têm uma ligação aberta ao LLM local. Nunca lê o que enviam. Um plugin aparece com o seu próprio nome, um agente com o seu, e tudo o resto com o nome do programa. A própria app e o LLM Monitor, que só leem o estado, não contam.

> **NOTA** A luz só se vê enquanto o programa gera e apaga-se cerca de dez segundos depois do seu último pedido. Um programa aberto mas inativo — o Video Trans sem fala para traduzir — não mostra nada. Um programa continua no menu meio minuto depois do seu último pedido.

CAPÍTULO 6

Modelos

Clique direito num LLM local instalado › **Modelos…**, ou clique em **Adicionar modelo** em **Configuração**. A página tem uma lista à esquerda e o modelo escolhido à direita, em três separadores.

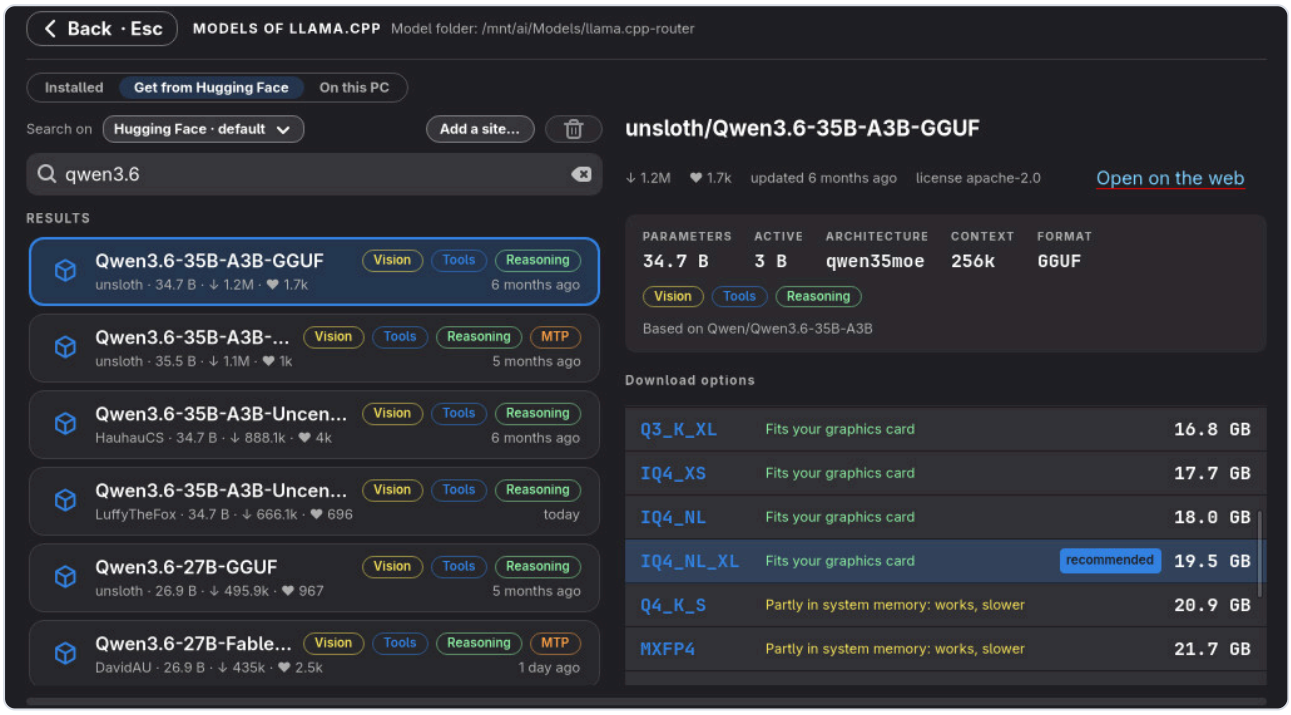


Figura 9 · Obter do Hugging Face: o que um modelo sabe fazer e que tamanho cabe na sua placa gráfica.

6.1 Obter do Hugging Face

Explore os modelos de chat GGUF mais populares ou pesquise. Cada resultado mostra autor, tamanho, transferências, gostos, idade e o que sabe fazer: **Visão**, **Ferramentas**, **Raciocínio**, **MTP**. A página do modelo mostra parâmetros, arquitetura, contexto, licença e a ficha do modelo. **O que significa o nome** explica as partes do nome, como *35B-A3B* ou *Q4_K_M*.

Opções de transferência lista cada tamanho com o modo como cabe: na placa gráfica, em parte na memória (funciona, mas mais devagar) ou provavelmente demasiado grande. A melhor escolha já está feita; **Descarregar** traz-a para a pasta dos modelos. Para o Ollama, que transfere pelo nome do tamanho, só são oferecidos os tamanhos que consegue transferir: um ficheiro dividido, um numa subpasta ou um que o Hugging Face recusa para o Ollama aparece esbatido, com o motivo, e nunca é recomendado.

As transferências correm em segundo plano e aparecem numa faixa no topo da página principal. Se a ligação cair, tentam de novo e continuam. Se a app foi fechada durante uma transferência, esta aparece como interrompida, com **Continuar** ou **Descartar**.

6.2 Transferências interrompidas na pré-visualização Flutter

Na pré-visualização Flutter, uma transferência de modelo ou um pull do Ollama cuja ligação cai é repetida após 5, 15, 30, 60 e 120 segundos e continua onde parou. Se a app terminar antes, o arranque seguinte mostra um aviso com cada transferência interrompida, onde parou, **Continuar** (segue com a parte já transferida) e **Descartar**. Só se guarda como a voltar a iniciar, em `~/.local/state/maprisma/jobs.json`, legível só por ti; uma transferência que ainda corre noutro lado não é iniciada duas vezes.

6.3 Perguntar a um agente que modelo serve

Não sabe que modelo escolher? Quando há um agente para as verificações de IA (Definições › **Agente para as verificações IA**), o cartão **Perguntar a um agente** por baixo da pesquisa aceita uma pergunta com as suas próprias palavras — por exemplo *um modelo de programação com contexto longo que caiba na minha placa gráfica*. **Perguntar** (ou `Ctrl`+`Enter`) envia-a, junto com a placa gráfica, a sua memória, a memória do computador e os modelos que já tem, a esse agente, que procura na web; enquanto trabalha, aparecem um indicador animado e o tempo decorrido. Ao fim de um ou dois minutos, o cartão mostra o seu breve resumo, e os modelos sugeridos aparecem por baixo na lista de resultados, com o título *Sugerido por …*: cada um encontrado no Hugging Face, com o seu tamanho e porque serve; um clique abre a sua página e as opções de transferência como em qualquer resultado de pesquisa. Uma sugestão que o Hugging Face não tem é indicada por baixo do título. Sugere até 10 modelos; **Sugestões**, à direita da linha do agente, define outro número (5, 10, 15 ou 20). Usa os tokens do agente; a pergunta não é guardada, e a página de cada LLM local pergunta por si.

6.4 Instalados

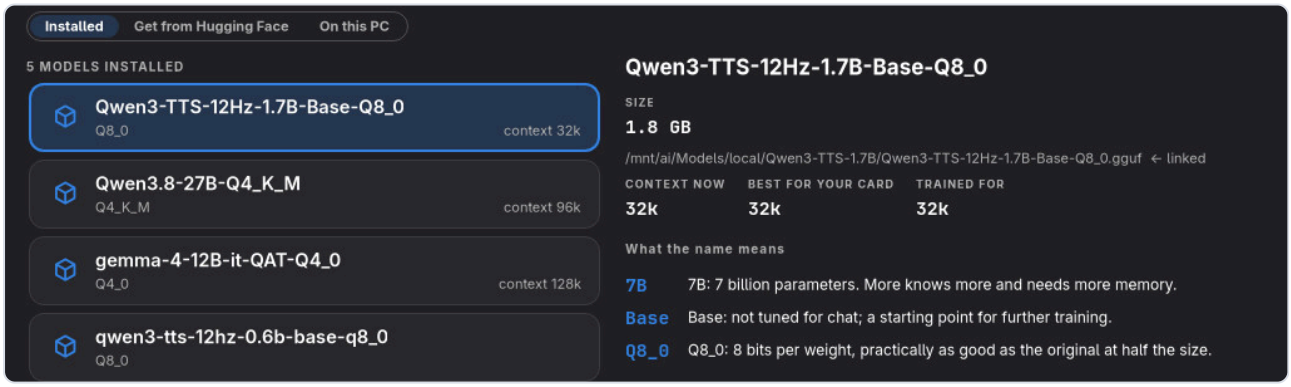


Figura 10 · Instalados: contexto atual, o melhor para a sua placa e o do treino; o que significa o nome; Remover.

Cada modelo mostra tamanho, ficheiro e contexto — *atual*, *o melhor para a sua placa* e *treinado para*. **Remover** pergunta primeiro e move os ficheiros para o lixo, de onde os pode restaurar. Um modelo que é só uma ligação perde apenas a ligação.

6.5 Tamanho do contexto

O contexto é quanto texto o modelo consegue ter presente de uma vez. A app calcula, a partir do próprio ficheiro do modelo, o maior contexto que cabe na sua placa gráfica. **Definir contexto** escreve-o nas definições do LLM local, com cópia de segurança e Desfazer. Os agentes precisam de um

contexto bastante grande; quando liga um agente a um modelo do LM Studio sem contexto definido, o melhor é definido no mesmo passo.

6.6 Neste PC

Lista ficheiros GGUF que o LLM local ainda não lê — na sua pasta dos modelos, nas pastas dos outros LLMs locais, em `~/Downloads` e em `~/Models`. **Adicionar** torna um deles modelo seu sem uma segunda cópia (através de uma ligação), exceto no Ollama, que importa a sua própria cópia.

6.7 Mover e eliminar modelos

Clique direito num modelo do llama.cpp, KoboldCpp ou LM Studio › **Mover para…**, ou arraste a caixa do modelo para a pasta de outro LLM local. Primeiro vê quanto é movido e que agente usa o modelo. Desfazer volta a mover os ficheiros. **Eliminar…** pergunta primeiro e move os ficheiros para o lixo.

6.8 Uma pasta dos modelos

Definições › **Pasta dos modelos** define uma pasta para os modelos de todos os LLMs locais. **Arrumar modelos…** mostra antecipadamente como os ficheiros seriam ordenados (`<pasta>/<editor>/<modelo>/`), e nada muda até premir **Arrumar**. Com **Os caminhos antigos continuam a funcionar** (ligado por omissão) fica uma ligação em cada local antigo. **Desfazer a última arrumação** repõe tudo.

> **ATENÇÃO** Ao mover para outro disco, os modelos são primeiro copiados e os originais só são removidos quando a cópia está completa. Certifique-se de que há espaço livre suficiente no destino; caso contrário, a app recusa a mudança.

CAPÍTULO 7

Ligar desenhando

Prima numa caixa do mapa e arraste até outra para as ligar. Cada alteração mostra, antes de escrever, exatamente que linhas de que ficheiro mudam, faz uma cópia de segurança e pode ser desfeita a partir da barra que aparece a seguir.

7.1 Agente ↔ LLM local

Arraste de um agente para um LLM local ou ao contrário. Escolha o modelo e se o agente o usa logo. Para alterar uma ligação, selecione o seu agente (ou clique na linha): cada linha que pode ser movida ou removida mostra um **gancho laranja** na ponta. Arraste o gancho para outro LLM local — ou para o vazio para a remover; ao passar sobre um gancho vê o que faz arrastá-lo. Com um agente selecionado, um clique num modelo de um LLM local ligado torna esse modelo o modelo do agente.

Agentes suportados: Hermes, Pi, Oh My Pi, OpenCode, OpenClaw e DeepSeek Harness.

7.2 Agente ↔ servidor MCP

Arraste de um agente para um servidor MCP. As definições do servidor são copiadas para o formato próprio desse agente. Suportados: Claude Code, Codex, Copilot, Crush, Cursor, Gemini, Goose, Grok, Hermes, Muse, Oh My Pi, OpenClaw, OpenCode e Pi. Os ficheiros de definições que podem conter chaves são sempre escritos de modo que só o utilizador os possa ler.

7.3 Agente ↔ pasta de skills

Arraste de um agente para uma pasta de skills. O Pi e o Hermes recebem a pasta nas suas definições; qualquer outro agente recebe uma ligação por skill na sua própria pasta de skills. Uma pasta que um agente lê por si (a maioria lê `~/.agents/skills`) não pode ser retirada só a esse agente.

7.4 Encontrado neste PC

Sempre que a janela abre (e com **Atualizar**), a aplicação procura servidores MCP e pastas de skills neste computador, sem iniciar nada: programas que oferecem um servidor MCP em `~/.local/bin`, `~/Applications`, `/opt` e nas pastas adicionadas na Configuração, e pastas de skills trazidas por aplicações ou que estão numa pasta adicionada. O que nenhum agente usa ainda aparece no mapa como uma caixa esbatida sem linhas. Nada é ligado sozinho.

1. Arraste de um agente até à caixa esbatida.
2. A página de ligação mostra como o servidor será iniciado — o comando que o seu próprio texto de ajuda indica, por exemplo com uma pasta que pode ler e escrever — e exatamente o que muda nas definições do agente.
3. **Ligar** escreve-o. A caixa deixa de estar esbatida, e a aplicação verifica logo se o servidor responde (ver Quando uma ligação falha).

O servidor MCP de uma aplicação funciona sozinho: a janela da aplicação não precisa de estar aberta. Os programas dentro de uma AppImage ainda não extraída não estão no mapa; **Adicionar servidor MCP** › **Neste PC** extrai-os primeiro.

> **DICA** Guarde as aplicações com um servidor MCP e as suas próprias skills numa pasta cada — por exemplo num disco partilhado por vários sistemas — e adicione essas pastas na Configuração. Noutro computador, adicione lá as mesmas pastas, e tudo o que contêm também aparece no seu mapa.

7.5 Ligações locais na pré-visualização Flutter

Abra Ligações e Adicionar servidor MCP. Indique um nome, escolha stdio, HTTP ou SSE e os agentes instalados. Em stdio, separe o executável e ponha um argumento por linha; os espaços na linha ficam nesse argumento. Adicione variáveis ou cabeçalhos como pares nome/valor; os valores ficam ocultos. A pré-visualização mostra destinatários e contagens de alterações, sem valores da configuração. A substituição de definições com o mesmo nome é indicada antes de confirmar. Aplicar usa cópia de segurança e Desfazer. O Maprisma não inicia nem testa o servidor aqui; os agentes podem iniciá-lo ao ler a configuração.

Adicionar competências escolhe uma pasta existente com pastas de competências, cada uma com SKILL.md. Escolha a pasta principal e os agentes e reveja ligações ou definições. Os originais ficam no lugar; as ligações acompanham alterações posteriores. As competências podem conter scripts executados pelos agentes. Pastas nativas já lidas não exigem nova ligação. A pesquisa verifica 200 entradas por pasta, um nível de categorias e 1.000 entradas no total; escolha uma pasta menor se necessário. Uma pré-visualização admite 32 agentes e 200 ligações. Desfazer preserva alterações posteriores. Catálogos online, transferências e testes de ferramentas MCP ficam fora destes formulários locais.

7.6 Encontrados neste PC na pré-visualização Flutter

Na pré-visualização Flutter, a aplicação procura ao arrancar, em segundo plano e sem executar nada, programas deste PC que oferecem um servidor MCP (os seus ficheiros contêm as palavras próprias do protocolo; como iniciá-los vem do seu próprio texto de uso) e conjuntos de skills que as aplicações trouxeram. Os programas grandes são lidos uma vez e memorizados. Problemas por resolver indica **Novo neste PC** para o que ainda não foi mostrado; **Adicionar servidor MCP…** ou **Adicionar skills…**, ou **Encontrados neste PC** em Ligações, lista-os, os novos com uma ★. **Adicionar…** abre o formulário preenchido com o nome e o comando de arranque, ou a pasta, para verificar e alterar antes da pré-visualização. A procura em si nunca executa um AppImage. **Ver por dentro…** desempacota só os programas de cada um, fechados pelo bubblewrap (sem rede, sem pasta pessoal, só uma pasta temporária gravável), lê-os e apaga-os de novo; o que foi encontrado fica memorizado. Adicionar um servidor que vive num AppImage pergunta primeiro se deve desempacotar todo o AppImage em `~/Applications/<app>-<version>/` (**Desempacotar**), da mesma forma. Um AppImage mais recente de uma aplicação já vista por dentro ou desempacotada é só nomeado (**Nova versão encontrada**): um ficheiro com um nome mais recente não prova uma atualização autêntica, por isso olha-se para dentro dele como para qualquer outro.

Quando uma aplicação é encontrada numa versão mais recente enquanto agentes, a lista de servidores MCP ou uma ligação em `~/.local/bin` ainda iniciam a antiga, a lista mostra-a com as suas versões e quem a usa. **Atualizar…** (depois de desempacotar o AppImage mais recente, se for um) pré-visualiza uma só alteração para tudo: cada entrada recebe o novo programa (os argumentos para a pasta antiga mudam com ele), as ligações apontam para a nova pasta e a pasta antiga vai para a cópia de segurança. Fica lá 7 dias; Desfazer repõe tudo.

7.7 Quando uma ligação falha

Logo depois de uma ligação ser escrita, a app verifica se vai funcionar: se um LLM local está instalado e oferece o modelo, se um servidor MCP responde, se as ligações das skills estão intactas. Uma ligação falhada fica no mapa como **linha vermelha tracejada** com um **!**; uma só com avisos é **amarela tracejada**. Clique na linha ou no seu **!** para ver sugestões:

- **Testar de novo** volta a verificá-la; se passar, a linha volta ao normal.
- **Tentar de novo…** abre a página de ligação para uma ligação que nunca foi escrita.
- **Remover ligação** remove-a.

CAPÍTULO 8

Servidores MCP

Os servidores MCP dão ferramentas extra aos agentes. **+ Servidor MCP** no mapa ou por baixo dos cartões abre a página para adicionar: uma lista à esquerda, a página do servidor escolhido à direita.

8.1 De onde vêm os servidores

Separador	Conteúdo
Recomendados	Os servidores de referência do projeto MCP (Filesystem, Fetch, Git, Memory, Sequential Thinking, Time, Everything) e servidores oficiais de empresas conhecidas (GitHub, Playwright, Chrome DevTools, Brave Search, Context7), agrupados por finalidade e fixados na versão mais recente.
Registo MCP	Uma pesquisa no registo MCP oficial. Cada resultado indica o seu editor tal como o registo o verificou. A pesquisa pode demorar dez segundos ou mais. Os resultados vêm de 20 em 20; a linha por baixo da lista percorre-os e mostra onde está, por exemplo **‹ 20 anteriores** · 21 – 40 / 200 · **Próximos 20 ›** (com */ 40+* quando o registo não diz quantos há).
Neste PC	Programas neste computador que oferecem um servidor MCP, encontrados lendo os seus ficheiros — nada é iniciado. Só se olha para dentro dos AppImages quando clica em **Ver por dentro…**; são descompactados fechados pelo bubblewrap (sem rede, sem pasta pessoal), por isso o `bwrap` tem de estar instalado.
À mão	Uma linha de comandos ou um endereço web com cabeçalhos.

Não sabe de que servidor precisa? No separador **Registo MCP**, o cartão **Perguntar a um agente** por baixo da pesquisa aceita um pedido com as suas próprias palavras — por exemplo *ler e escrever as minhas páginas do Notion*. **Perguntar** (ou `Ctrl`+`Enter`) faz o agente para as verificações de IA transformá-lo em algumas palavras de pesquisa; a app pesquisa com elas na lista do registo que transferiu, como a pesquisa normal, e o agente escolhe entre o que foi encontrado os servidores que servem, cada um com o motivo. Não pesquisa na web, só vê o seu pedido e os textos públicos do registo e só pode escolher servidores que estão mesmo na lista. Enquanto trabalha, aparecem um indicador animado e o tempo decorrido; após alguns segundos a um minuto, as sugestões aparecem na lista de resultados, até 10 (**Sugestões** à direita da linha do agente: 5, 10, 15 ou 20). Uma pesquisa escrita depois substitui-as.

Neste PC também procura nas pastas adicionadas com **Adicionar pasta…** em Configuração › **Servidores MCP locais**: em cada pasta propriamente dita e nas pastas de aplicações que contém, como `photocraft-0.16.0/usr/bin`.

8.2 Configurar um servidor

A página do servidor diz quem o fez e quão verificado está, o que faz (com coisas que pode pedir a um agente), como corre — como programa através de `npx`, `uvx` ou Docker, ou como serviço web — e do que precisa, como uma pasta ou uma chave de API com uma ligação para onde a obter. Por baixo, **Configurar**:

1. Preencha os campos de que o servidor precisa.
2. Assinale os agentes que o devem receber.
3. Clique em **Testar** para pedir ao servidor as suas ferramentas.
4. Clique em **Adicionar**: todos os agentes assinalados são escritos de uma vez, com pré-visualização e um Desfazer.

> **NOTA** Os valores secretos, como chaves de API, são passados no ambiente ou em cabeçalhos, nunca numa linha de comandos onde outros utilizadores do computador os possam ler. Um servidor que precisaria de um segredo na sua linha de comandos tem de ser configurado à mão.

8.3 Os servidores MCP na pré-visualização Flutter

Na pré-visualização Flutter, **Encontrar servidores MCP…** em **Ligações** abre os separadores **Recomendados** (fixados na versão mais recente ao abrir) e **Registo MCP** (20 resultados de cada vez com **‹ 20 anteriores** e **Próximos 20 ›**; a lista do registo é transferida em segundo plano para uma pesquisa mais rápida). A página de um servidor diz quem o fez, o que faz, de que precisa e **Como corre**; a forma que pode correr neste PC aparece como **recomendado**, e um `npx`, `uvx` ou Docker em falta é indicado. Preenche os campos de que precisa (os secretos ficam ocultos), verifica o **Nome**, marca os agentes e clica em **Pré-visualização**: a pré-visualização, **Aplicar** e **Desfazer** são os de uma alteração de ligação. Só os valores que escreveste vão para o serviço; o comando vem da entrada do servidor e é verificado lá outra vez. Um segredo que ficaria numa linha de comandos é recusado; um servidor assim configura-se à mão.

Testar inicia uma vez a forma escolhida (ou consulta o seu endereço) e lista as ferramentas que oferece, antes de se escrever o que quer que seja. Um servidor já configurado mostra **Ferramentas** no seu cartão em **Ligações**: a última resposta com a data e **Perguntar ao servidor** (ou **Perguntar de novo**), que o inicia uma vez como o seu primeiro agente faz e depois o para. Os seus **Prompts** também aparecem. A resposta fica guardada para a próxima vez; as chaves das suas definições vão em privado para o processo que pergunta e nunca são mostradas.

8.4 Gerir servidores

ⓘ **Detalhes** no cartão de um servidor MCP mostra o ficheiro de definições de cada agente, o comando ou o endereço e, se pedir, as ferramentas do servidor; os valores das chaves nunca são mostrados. **Remover** retira-o de todos os agentes. Os servidores ficam na lista quando nenhum agente os usa, para que uma linha os possa voltar a ligar.

Quando aparece em `~/Applications` uma versão mais recente de uma app que fornece um servidor MCP, uma sugestão diz isso. **Atualizar…** desempacota a nova versão, aponta cada agente para ela e guarda a antiga sete dias como cópia de segurança. **Procurar atualizações** (em **Neste PC**) faz isto

para todas essas apps de uma vez depois do seu clique: olha para dentro das novas versões, oferece todas as atualizações numa pré-visualização com uma cópia de segurança e um Desfazer, e depois testa cada servidor atualizado; uma sugestão indica qualquer um que não responda.

CAPÍTULO 9

Skills

Uma skill é uma pasta de instruções que ensina a um agente uma tarefa concreta. **+ Skills** abre uma página organizada como a dos MCP.

9.1 Encontrar skills

- **Neste PC**: pastas de skills que as apps trouxeram e que nenhum agente lê ainda.
- **Da web**: uma pesquisa no diretório skills.sh, noutro site com a mesma API ou num repositório GitHub de skills. Uma skill é obtida do seu repositório GitHub para uma pasta à sua escolha, por omissão `~/.agents/skills`.

> **ATENÇÃO** As skills podem conter scripts que um agente pode executar mais tarde. A página mostra o repositório e lista os scripts antes de obter o que quer que seja — veja-os.

Neste PC também procura nas pastas adicionadas com **Adicionar pasta…** em Configuração › **Skills**.

9.2 Skills da web na pré-visualização Flutter

Na pré-visualização Flutter, **Encontrar skills na web…** em **Ligações** pesquisa no skills.sh. Um resultado indica o seu repositório GitHub e quantas vezes foi instalado; escolhê-lo transfere os ficheiros da skill — no máximo 150 ficheiros e 30 MB, sem ligações simbólicas, nada fora da sua pasta — sem executar nenhum. O cartão mostra a descrição, quantos ficheiros tem e que scripts um agente poderia executar mais tarde. Escolhe a pasta para onde vai (**Em**: a partilhada `~/.agents/skills` ou uma das tuas pastas de skills), marca os agentes em **Também para** e clica em **Pré-visualização**: os ficheiros novos são listados, e **Aplicar** escreve-os com uma cópia de segurança, pelo que **Desfazer** os retira de novo. Uma skill cuja pasta já existe não é substituída. Até lá, os ficheiros transferidos esperam numa pasta privada; cada um é comparado com o que a pré-visualização mostrou antes de ser escrito.

9.3 Pastas de skills

Cada cartão de pasta de skills mostra as suas skills e os agentes que a leem. Uma pasta partilhada através de ligações aparece uma só vez, onde realmente está. ⓘ **Detalhes** lista cada skill com a sua descrição e como cada agente recebe a pasta.

CAPÍTULO 10

Definições e ajuda

10.1 Temas e luzes do mapa Flutter

No Flutter, Definições oferece Sistema, Claro e Escuro com pequenas pré-visualizações. Sistema segue a aparência e a paleta do ambiente de trabalho; Claro e Escuro fixam o brilho com fundos adequados e cores legíveis. A escolha é guardada e aplica-se também aos cartões de tarefas. A preferência de aparência GTK mantém-se separada. Com Sistema, um novo tema do ambiente de trabalho aparece em dois segundos, também enquanto a aplicação está ocupada com uma transferência, uma instalação ou uma alteração.

A taxa das luzes oferece 20 fps (predefinido), 30 e 60. É o máximo para as luzes animadas, não o intervalo de análise automática. A velocidade do movimento mantém-se; taxas superiores acrescentam fotogramas intermédios. A taxa real depende do ecrã e do tempo de renderização disponível. Luzes desligadas, movimento reduzido, janela sem foco ou mapa oculto continuam a parar a animação. A escolha é guardada.

As definições oferecem também **Iniciar LLMs locais com os seus agentes**: quando um agente ligado a um LLM local inicia, o seu LLM inicia também, e quando o último desses agentes fecha, o LLM para e a sua memória gráfica fica livre (um serviço de utilizador faz isto, também com a janela fechada). **Agente para as verificações IA** escolhe que agente instalado é oferecido primeiro para a ajuda de IA e o Smart Check do mapa do sistema; para isso são usados os seus tokens. A lista mostra os agentes que o mapa do sistema encontrou capazes de o fazer.

10.2 A janela flutuante sob a barra no Omarchy

No Omarchy a aplicação Flutter abre como janela flutuante: uma janela no canto superior direito sob a barra, sem barra de título, como as janelas dos plugins. `Esc` na página principal ou um clique fora fecha-a; iniciar a aplicação de novo (ou o ícone da barra, que executa `maprisma --toggle`) trá-la de volta ao seu lugar, com tudo como a deixou. Um seletor de ficheiros da aplicação não a fecha. Configurações › **Janela** escolhe **Automático: uma janela flutuante sob a barra no Omarchy** (predefinido), **Uma janela flutuante sob a barra** ou **Uma janela normal**; uma alteração aplica-se da próxima vez que a janela abrir. Noutros ambientes de trabalho a aplicação é uma janela normal.

10.3 Dados de utilização e cantos do Omarchy na pré-visualização Flutter

Quando na pré-visualização Flutter se abrem os detalhes de um agente cuja utilização o Omarchy recolhe (plano, limites e quando reiniciam), a aplicação pede ao Omarchy dados novos em segundo plano se os últimos tiverem mais de dez minutos ou forem anteriores ao último início de sessão do agente, e depois mostra-os; no máximo uma vez a cada dez minutos por agente. Configurações › **Cantos** define os cantos dos cartões, das caixas de diálogo e das caixas do mapa: **Como as janelas** segue o arredondamento das janelas do Hyprland (com o número de píxeis), **Arredondados** ou **Retos**.

10.4 Definições

A roda dentada (`Ctrl`+`,`) abre as definições. `Retrocesso` ou `Esc` volta atrás.

Definição	O que faz
Idioma	O idioma do sistema ou English, Deutsch, Français, Español, Italiano, Português, Nederlands, Polski.
Aparência OMARCHY	As cores do tema Omarchy ou a aparência Adwaita do GNOME com o modo claro/escuro e a cor de destaque do sistema.
Animações do mapa	Uma luz percorre as linhas do mapa: **Agentes e outros programas** (a linha de um agente a trabalhar e a linha de um LLM local até um modelo que outro programa usa), **Só os meus agentes** ou **Desligado**. Um pequeno mapa animado por baixo da definição mostra o que cada escolha move.
Caixa de estatísticas de uso	Os números ao longo da linha de um agente a trabalhar (ver Trabalhar com o mapa): ligada ou não, quais de **Tokens por segundo**, **Tamanho do contexto** e **Contexto usado** mostra, e o seu **Tamanho do texto** (7 pt por predefinição).
Subagentes	Para a página Subagentes: quanto tempo ficam os subagentes terminados (**30 minutos** por omissão; 10, 30, 60 ou 120) e quantos no máximo (**10** por omissão; 5, 10, 20 ou 40). Os que trabalham são sempre mostrados.
Custos	**Moeda**: dólar americano, ou euro à taxa de referência diária do Banco Central Europeu (obtida uma vez por dia de ecb.europa.eu). **Limite diário** para um agente pago por token (**100** por omissão; de 10 a 500). **Amarelo a partir de** e **Vermelho a partir de**: a parte usada a partir da qual uma indicação de custo ou uma bateria fica amarela (75 % por omissão) e vermelha (90 %). **Avisar no vermelho** (ligado por omissão): uma notificação no ambiente de trabalho quando um agente fica vermelho — uma vez por cada limite até este ser reposto, ou uma vez por dia para um custo.
Separadores de terminal acima do mapa	Um separador por cada terminal de agente dentro da janela.
Cantos	Como as janelas, arredondados ou retos.
Iniciar LLMs locais com os seus agentes	O vigilante em segundo plano de Iniciar e parar com os agentes.
No lançador	A entrada do lançador; desligado remove-a.
Os agentes arrancam em	A pasta em que os agentes arrancam.
Funções de IA: **Pesquisar agentes novos**	Perguntar a um agente instalado onde um agente desconhecido guarda as suas definições (ligado por omissão).
Funções de IA: **Verificações IA diárias**	Uma vez por dia, um agente procura na web novas versões dos seus servidores MCP (desligado por omissão; usa os tokens do agente). Ver Smart Check.

Definição	O que faz
Funções de IA: **Agente para as verificações IA**	Claude Code ou Codex: faz as verificações IA e as pesquisas e assume as tarefas **Fazer com …** e **Corrigir com …**.
Funções de IA: **Ajudante local**	Um agente com um modelo do seu LLM local para a ajuda IA que não precisa da web (desligado até escolher um). Ver O ajudante local.
Pasta dos modelos	Uma pasta para todos os modelos, com **Arrumar modelos…**.
Complementos MAPRISMA	**Procurar complementos online**, a página de complementos e **Configuração**.
Sobre	A versão, o autor e a licença; **Site** abre o projeto no GitHub e **Comunicar um problema** a sua lista de problemas.

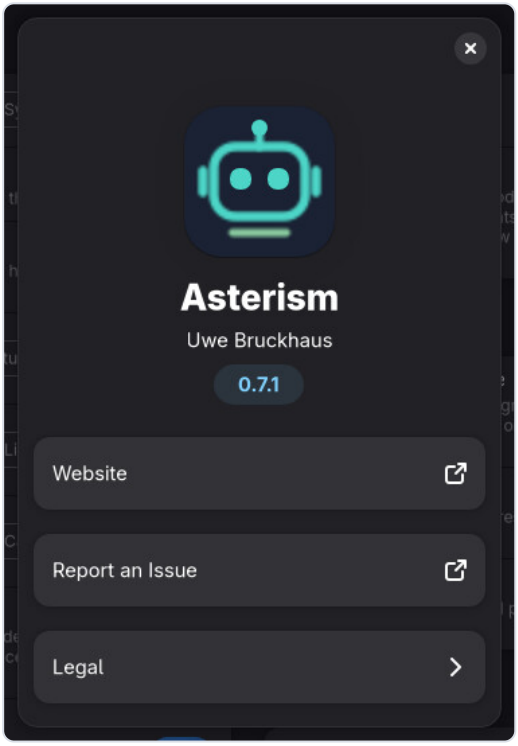


Figura 11 · Definições › Sobre.

10.5 Limites a vermelho na pré-visualização Flutter

Na pré-visualização Flutter, a aplicação verifica uma vez por minuto os limites do plano dos seus agentes instalados (uma sessão de 5 horas, uma semana…). Quando um está gasto até ao nível vermelho de Definições › Custos, uma notificação no ambiente de trabalho di-lo, com a parte gasta e quando o limite é reposto: uma vez por limite até ser reposto, também depois de reiniciar. **Avisar no vermelho** nas Definições desliga-a. O custo de um agente pago por token ainda não é mostrado na pré-visualização, por isso também não a sua notificação.

10.6 Alterações recentes e Desfazer após um reinício

Na pré-visualização Flutter, as Configurações terminam com **Alterações recentes**: as últimas 20 alterações que esta aplicação escreveu nas definições dos agentes (ligações, servidores MCP, skills, definições de segurança, atualizações de aplicações), a mais recente primeiro, com os ficheiros afetados. **Desfazer** repõe o que não voltou a ser alterado desde então — também depois de a aplicação ter sido fechada ou ter falhado, quando o **Desfazer** da última alteração já não existe. Uma alteração cujos ficheiros voltaram a ser editados não mostra **Desfazer**. Uma alteração que ficou a meio (a aplicação fechou durante a escrita) repõe apenas os ficheiros que já tinha escrito. As cópias de segurança ficam onde estão; Desfazer apenas as lê.

10.7 Ajuda

? (F1) abre a ajuda: **Abrir o manual** abre este manual (a edição que veio com a sua versão, no seu idioma) no seu visualizador de PDF; **Espaço em disco** (abaixo); uma animação sobre configurar desenhando; e o LLM Monitor, um plugin de barra complementar que mostra o que um LLM local está a fazer (uma pequena animação da sua entrada na barra e do seu painel). Ambas as animações são desenhadas nas cores do seu tema e seguem de imediato uma mudança de tema. A legenda ⓘ do cabeçalho explica cada ponto, linha e marca do mapa.

O **i** preenchido no cabeçalho abre **Informações**: cartões dos outros plugins do autor, lidos do Omarchy Plugin Marketplace quando a página abre, no máximo uma vez por hora (sem rede, a lista que esta versão conhece). Um clique num cartão abre a página do plugin no marketplace.

10.8 A ajuda na pré-visualização Flutter

Na pré-visualização Flutter, **?** no cabeçalho (ou F1) abre a **Ajuda**: **Abrir o manual** abre este manual no teu idioma (a cópia que veio com a app, senão a inglesa) no visualizador de PDF do ambiente de trabalho, e **Por onde começar** leva a **O seu primeiro agente**, **Procurar IA existente**, **Fornecedores e modelos** e **Ligações**.

10.9 Espaço em disco

Espaço em disco: agentes, LLM locais e modelos mostra quanto espaço ocupa a sua configuração de IA e onde. É medido em segundo plano sempre que a ajuda abre (de novo após dez minutos ou com **Medir de novo**) e só lido — nada é apagado daqui. O título mostra sempre os totais numa linha (*Agentes · LLMs locais · Modelos · No total*); um clique recolhe a secção. **Discos**, **Agentes**, **LLMs locais** e **Pastas de modelos** começam recolhidos, cada um com o seu total e os itens maiores; um clique abre-os.

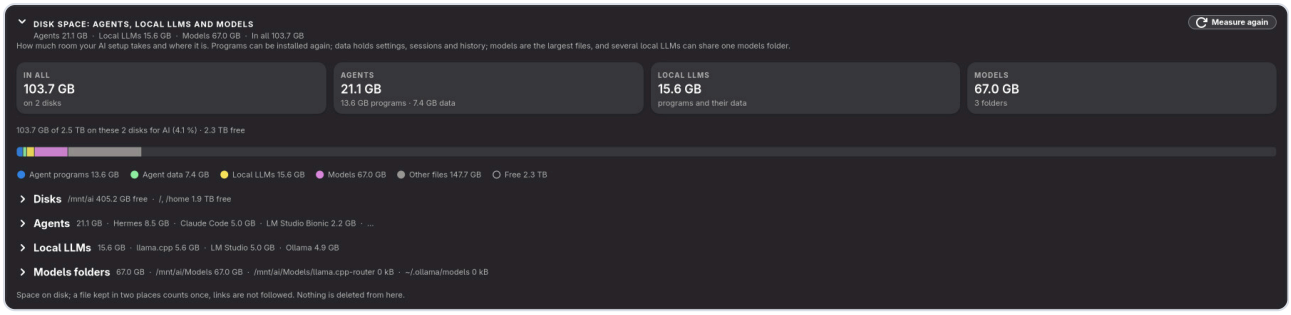


Figura 12 · Os totais, a parte da IA em todo o espaço em disco e as secções recolhidas com os seus totais.

- **Mosaicos**: tudo junto, depois programas dos agentes, dados dos agentes (definições, sessões, histórico), LLMs locais (programas e os seus próprios dados) e modelos.
- **Barra**: todo o espaço dos discos onde está a configuração de IA. As partes coloridas são a configuração de IA, o cinzento é tudo o resto nesses discos e o resto vazio está livre; a linha por cima dá a parte da IA em percentagem. Passe o rato pela barra para ver os tamanhos exatos.
- **Discos**: cada disco com o que a configuração de IA ocupa nele e quanto está livre.
- **Agentes**, **LLMs locais**, **Pastas de modelos**: uma linha cada, os maiores primeiro. Abra uma linha para ver cada pasta com tamanho e número de ficheiros e abri-la no gestor de ficheiros. Uma pasta de programa diz quantas versões são guardadas; para um agente com várias versões antigas no mise, uma dica menciona `mise prune`, que remove as que não são usadas. Uma pasta de modelos diz que LLMs locais a usam, ou que só contém ligações para modelos de outra pasta.

> **DICA** Os programas podem ser reinstalados e os modelos descarregados de novo; os dados dos agentes contêm as suas sessões e definições. Quando o espaço escassear, olhe primeiro para versões antigas de programas e para modelos que já não usa.

Os tamanhos são espaço em disco: um ficheiro guardado em dois sítios conta uma vez, e as ligações não são seguidas.

CAPÍTULO 11

Smart Check

O Smart Check deixa um agente online instalado — Claude Code ou Codex — examinar a configuração de IA deste PC, encontrar atualizações para os seus servidores MCP e fazer correções por si, e mostra quanto isso custou em tokens. Um ajudante local — um agente com um modelo do seu próprio LLM local — pode responder às perguntas que não precisam da web, neste PC e sem tokens. Cada execução mostra o seu progresso numa caixa por cima da janela e fica guardada no Protocolo. As duas estrelinhas do cabeçalho abrem-no; só aparecem enquanto um destes agentes estiver instalado. Definições › **Agente para as verificações IA** escolhe qual.

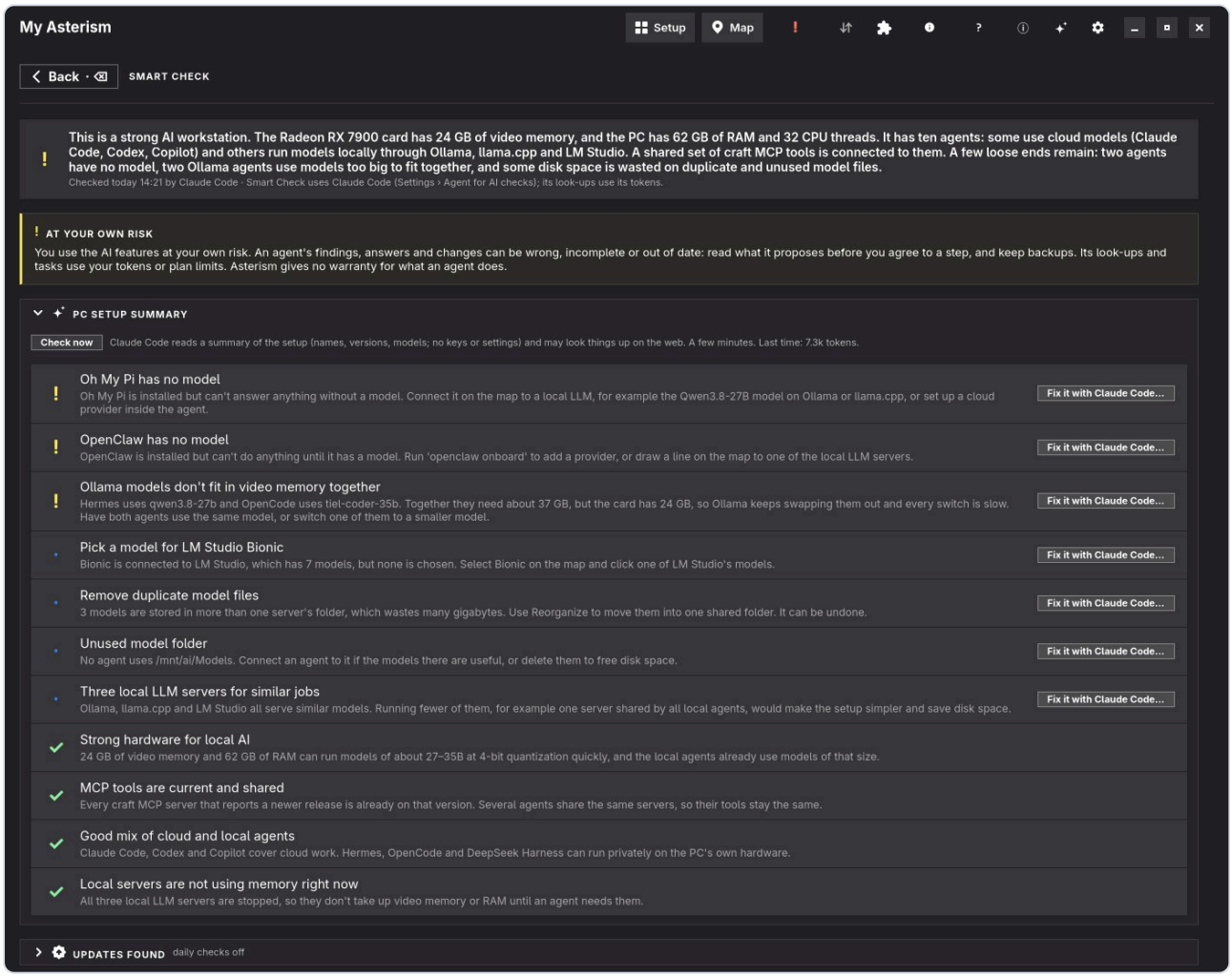


Figura 13 · O Smart Check depois de uma verificação pelo Claude Code: problemas (!), dicas (·) e o que está bem (✓).

> **POR SUA CONTA E RISCO** As conclusões, respostas e alterações de um agente podem estar erradas, incompletas ou desatualizadas. Leia o que propõe antes de aceitar um passo e mantenha cópias de segurança. As suas pesquisas e tarefas usam os seus tokens ou os limites do seu plano. A app não dá qualquer garantia sobre o que um agente faz. A mesma nota encabeça Definições › **Funções de IA**.

11.1 Resumo da configuração do PC

Verificar agora dá ao agente um pequeno texto sobre a configuração — a placa gráfica e a memória, os agentes com os seus modelos, os LLMs locais, os servidores MCP com as suas versões, as skills e o que Problemas por resolver encontrou — e deixa-o procurar coisas na web. Não contém chaves, cabeçalhos nem valores de definições. Demora um minuto ou mais em segundo plano; a linha por baixo do botão diz quantos tokens a última verificação usou. A resposta é um pequeno resumo e uma lista de resultados: **!** problemas, • dicas e ✓ o que está bem. As respostas que não se confirmam são deixadas de fora.

Corrigir com … ao lado de um problema ou de uma dica abre o agente num separador de terminal dentro da janela, com esse resultado como tarefa: verifica se é mesmo assim, explica e pergunta-lhe antes de cada passo que altere alguma coisa. A app inicia-o de forma a que tenha de perguntar — o Codex numa sandbox só de leitura que pede mais permissões, o Claude Code com cada comando e edição definidos para perguntar, diga o que disser a sua própria lista de permissões — e sem os seus servidores MCP. O resultado é-lhe entregue como dados a verificar, não como instruções. Passar servidores MCP para uma nova versão fica a cargo do **Atualizar** da app (abaixo), que altera as definições de todos os agentes de uma vez, com cópia de segurança e Desfazer.

11.2 Atualizações encontradas

Novas versões dos seus servidores MCP, de duas fontes:

- **As verificações IA diárias** (Definições › **Verificações IA diárias**, desligado por omissão): uma vez por dia o agente procura nas páginas oficiais de versões novas versões de servidores cuja versão a app consegue saber — apps descompactadas em `~/Applications/<app>-<version>/` ou como AppImage descompactado em qualquer outra pasta (o programa em `<folder>/<app>-<version>/usr/bin/`) e pacotes npm ou PyPI fixados numa versão. **Verificar agora** neste cartão pergunta de imediato, também enquanto as verificações diárias estão desligadas. Cada resultado tem **Página da versão** e **Fazer com …**: o agente descarrega o novo AppImage num separador de terminal, perguntando antes de cada passo.
- **A própria procura da app**: uma versão mais recente já está neste PC, mas um agente ainda inicia a antiga (o mesmo que a atualização no cartão do servidor em Configuração).

Quando a nova versão está neste PC, a sua linha di-lo e oferece **Usar…**: a pré-visualização da atualização mostra que agentes passam para ela; **Atualizar** altera-os todos, guarda a versão antiga com a cópia de segurança e testa cada servidor a seguir. Um servidor cujos agentes já iniciam todos a nova versão sai da lista.

> **DICA** Depois de uma ação iniciada aqui — uma atualização ou uma tarefa num separador de terminal —, **Voltar** leva-o de novo ao Smart Check, atualizado.

11.3 Enquanto um agente trabalha

Cada execução de ajuda IA que inicia — Verificar agora, Perguntar a um agente nas páginas Modelos e Adicionar servidor MCP, Verificar agora para as atualizações, Perguntar no cartão Desempenho de um LLM local — mostra uma caixa no meio da janela: a quem se pergunta (o agente e o seu modelo), os seus passos (um indicador animado que se transforma em ✓, ou em ✕ quando um falhou), o tempo decorrido e, no fim, os tokens que usou.

- **Em segundo plano** esconde a caixa; a execução continua e uma mensagem avisa quando termina.
- **Cancelar** termina o agente de imediato (o seu programa e tudo o que iniciou).
- Quando falha, a caixa fica com o que correu mal até a fechar; o texto pode ser selecionado e copiado.

As pesquisas na web (conselhos sobre modelos, atualizações MCP) correm com a web e mais nada: sem os servidores MCP, conectores, plugins e hooks do agente.

11.4 O ajudante local

Um agente ligado a um LLM local pode responder à ajuda IA com um dos modelos desse LLM: neste PC, sem web, sem tokens de um plano e sem nada sair do computador. Responde ao que não precisa da web: o resumo do Smart Check, que servidor MCP serve e as dicas no cartão Desempenho de um LLM local. Os conselhos sobre modelos e as atualizações de servidores MCP continuam a ir para o agente online, porque precisam da web.

A app sugere um — o maior modelo de chat que a sua placa gráfica comporta (os modelos de voz, de embeddings e de tokenizador ficam de fora) — no Smart Check, com **Usar**. Definições › **Funções de IA** › **Ajudante local** lista todos os agentes e modelos que o podem ser, primeiro o sugerido, e **Desligado**. Nada é usado até o escolher. Antes de responder, a app inicia o LLM local e carrega o modelo (um passo na caixa de progresso). O ajudante corre sem ferramentas, sem servidores MCP, sem extensões e sem ficheiros de projeto. Qualquer agente cujo complemento indique como lhe fazer uma pergunta offline pode ser o ajudante; o Pi é o primeiro.

11.5 Verificações com IA e o ajudante local na pré-visualização Flutter

Na pré-visualização Flutter, Configurações › **Funções de IA** tem **Verificações IA diárias** (desligadas por omissão) e, por baixo, o resultado da última verificação com **Verificar agora**, que pergunta de imediato, também com as verificações diárias desligadas. A verificação corre em segundo plano ao abrir a janela e é revista a cada hora; o agente das verificações com IA é consultado no máximo uma vez por dia (após uma pesquisa falhada, de novo ao fim de seis horas). O que encontra aparece em Problemas por resolver, em **Atualizações e sugestões**: **Página da versão** abre a publicação no navegador, **Fazer com …** mostra primeiro a tarefa tal como será enviada e só a inicia num separador

de terminal depois de **Continuar**, e **Usar…** abre o que foi encontrado neste PC quando a versão mais recente já cá está. Só são verificados os servidores cuja versão a aplicação consegue saber (ver Atualizações encontradas); os outros não são enviados.

O ajudante local escolhe-se em Configurações › **Funções de IA** › **Ajudante local**; a sugestão para a sua placa gráfica está marcada com **(sugerido)**. Depois de escolhido, é oferecido primeiro em **Perguntar** no cartão Desempenho de um LLM local.

11.6 Protocolo

Cada execução de ajuda IA é guardada, para que possa rever o que foi feito — ou pedir a alguém que conheça bem a configuração que o reveja. Cada entrada mostra quando correu, o agente e o modelo, o resultado (funcionou, falhou, cancelada), o tempo e os tokens. Aberta, mostra o texto completo: as opções que fecharam o agente, os passos, cada pergunta exatamente como foi enviada com a resposta do agente, o que a app alterou a partir da resposta (com a cópia de segurança, para Desfazer à mão) e, numa tarefa num separador de terminal, o que o terminal mostrava quando terminou. **Copiar** põe uma entrada na área de transferência para um relatório ou uma segunda opinião; **Abrir a pasta do protocolo** mostra os ficheiros.

O protocolo guarda as últimas 200 execuções em `~/.local/state/ai-agents-map/ai-help/`, legíveis só por si. As perguntas contêm nomes, números e definições, nunca chaves; o texto de um terminal contém o que estava no ecrã.

11.7 Estatísticas de tokens

Tokens usados hoje e nos últimos 30 dias, um gráfico por dia e os totais por agente e por plugin (outros plugins Omarchy que usam o mesmo assistente de IA também aparecem aqui). **Valor em API** é quanto as execuções custariam a preços de API, e a linha por baixo diz se a sua subscrição o cobre ou se é cobrado a uma chave de API. Uma tarefa num separador de terminal conta na sessão própria do seu agente; a página de detalhes do agente mostra esses tokens.

Mais abaixo, um cartão por tipo de tarefa — a verificação da configuração, atualizações de servidores MCP, a pesquisa de agentes novos, correções — com as suas execuções, tokens e os resultados mais recentes.

CAPÍTULO 12

Cartões de grupo e complementos no mapa

12.1 Expandir uma categoria

Na pré-visualização Flutter, cada coluna de IA tem um cartão de grupo. O contorno tracejado indica que nada está configurado; a contagem vem da deteção local. Os marcadores são apenas visuais, nunca nós ligados ou dados para IA. Use Expandir, duplo clique ou Enter no cartão focado. O grupo aparece sozinho ao centro. Voltar, Esc, Backspace ou o fundo restaura zoom e seleção. O mapa do sistema usa os mesmos controlos para hardware, discos, programas, ferramentas e pastas.

12.2 Gerir complementos

Os grupos mostram Ativos, Desativados, Encontrados neste PC e Não instalados conforme o conteúdo. O menu ou clique direito oferece ativação, detalhes, versão assinada mais recente ou início de sessão do fornecedor através de um agente instalado. Detalhes explica leituras, envios e capacidades. As alterações são imediatas com Desfazer e bloqueadas durante sessões de agentes desta aplicação ou uma instalação ativa. Agentes noutros terminais não bloqueiam os complementos. Procurar complementos online carrega explicitamente o catálogo assinado; expandir não o carrega. Reveja cada instalação ou atualização. Os novos complementos ficam desativados. Remover um complemento de utilizador cria uma cópia e preserva programas e dados; pode desfazer até fechar a aplicação. Os incluídos pertencem ao pacote.

Após uma remoção confirmada, cópias com mais de 30 dias podem ser limpas. Cópias com Desfazer ativo ficam; ligações, pastas não relacionadas e de outros proprietários são ignoradas. Desfazer dura até fechar a aplicação. Na primeira utilização, Agentes em Boas-vindas reabre o mesmo grupo com Ativar / Mais tarde fixo em baixo; não termina as boas-vindas nem abre uma vista normal.

Instalar a partir de ficheiro… usa um extra .zip que tu próprio descarregaste. Decide a sua própria assinatura, como numa transferência do catálogo: o ficheiro é verificado primeiro e mostrado como uma entrada do catálogo (nome, versão, tipo, o que lê e envia, e se substitui uma versão instalada), e nada é instalado até confirmares. Então o ficheiro é lido de novo e tem de ser ainda o que reviste. Por enquanto, na pré-visualização Flutter só se podem usar extras de agentes e de fornecedores.

12.3 Primeira utilização e funções existentes

Boas-vindas aparece sobre o mapa com Examinar este PC / Ignorar e uma ligação ao mapa do sistema. Monte discos existentes antes de pesquisar. Agentes abre com as deteções marcadas e Ativar / Mais tarde sempre visível em baixo. Ativar é uma ação conjunta com um Desfazer; Mais tarde preserva escolhas. Outros tipos já ativos são mantidos. Os arranques seguintes não expandem automaticamente. Modelos abre transferência/importação e reutilização de pastas; MCP e competências abre Ligações. Novas definições MCP e novas pastas de competências continuam por migrar. O foguete abre o guia do primeiro agente. Os grupos substituem as páginas Configuração e Complementos.

CAPÍTULO 13

Editar o mapa de IA Flutter

Arraste um agente para um fornecedor, servidor MCP ou conjunto de competências, ou desenhe no sentido contrário. Selecione um agente e depois um modelo de um fornecedor ligado. Desenhar do fornecedor para o seu modelo abre a seleção de agentes; fornecedores locais compatíveis também podem carregar sem agentes. Cada alteração passa por Pré-visualização, Aplicar, progresso e resultado com Anular. Cancelar não escreve nada.

Selecione uma caixa para mostrar as pegas laranja. Arraste uma pega para outro fornecedor ou modelo compatível, ou para espaço vazio para desligar. A pega de modelo do fornecedor muda todos os utilizadores configurados, ou desliga-os e descarrega quando suportado. Um motor ocupado recusa o descarregamento. Fornecedores online usam o comando registado de início ou fim de sessão num terminal após revisão.

Passar o ponteiro realça os vizinhos. Shift-clique acrescenta caixas, com no máximo um agente. Fixar mantém o realce. Recolha um cartão com a seta, arraste o cabeçalho ou use o menu. Arraste a pega de ordem ou escolha Mover para cima / Mover para baixo. Disposição e fixações guardam-se separadamente das ligações; cartões de tarefas conservam seleção e zoom.

Com o teclado, abra Ações, escolha Ligar ou Mover ligação e ative o destino. Desligar remove a ligação pendente; Cancelar ou Esc abandona. Setas, menus de movimento, Fixar e opções de ordem substituem o arrastar. Abrir mostra a gestão existente. Os botões iniciam na pasta predefinida, mostram ou param uma consola própria, ou permitem parar uma instância externa após revisão.

Todas as luzes, Luzes dos agentes e Luzes desligadas regulam a atividade estimada. Param sem linhas ativas, com mapa oculto, sem foco ou com movimento reduzido. Linhas com falhas mostram indicações limitadas com Reparar ou Desligar. Anular restaura configuração e metadados de falhas; não reverte carregamento, descarregamento nem início ou fim de sessão. Uma operação parcial conserva a cópia e Anular.

CAPÍTULO 14

Cartões de tarefas no mapa

Ligações, ações de agentes, remoção de modelos e alterações de extensões abrem um cartão de tarefa na pré-visualização Flutter. Com uma área de mapa de pelo menos 900 píxeis lógicos e elementos conhecidos, surge à esquerda um mapa só de leitura. Mostra esses elementos e os vizinhos diretos, com linhas para o cartão. As colunas vazias são omitidas. O mapa principal mantém ampliação, posição e seleção. A vista geral dos fornecedores ocupa toda a largura; as ações de um motor mostram meio cartão quando os seus elementos no mapa são conhecidos.

Reveja as alterações antes de confirmar. O mesmo cartão mostra progresso e resultado; use Desfazer quando disponível. Voltar ou Esc cancela uma revisão ou fecha um cartão concluído. Não fecham tarefas em curso; apenas operações canceláveis mostram Cancelar. Pesquisa e transferência de modelos, catálogo e guia inicial ocupam toda a largura. O mesmo acontece em janelas estreitas e tarefas sem elementos conhecidos. Definições, terminais, utilização e detalhes de subagentes mantêm as vistas existentes.

CAPÍTULO 15

Mapa do sistema

15.1 Primeira utilização e discos separados

Pré-visualização de desenvolvimento Flutter: os mapas de IA e do sistema mantêm zoom e seleção. Monte os discos existentes antes de Examinar este PC. A pesquisa limitada lê metadados locais sem executar programas detetados nem enviar dados. Ignorar mantém a pesquisa nos discos desligada. Hardware, discos, agentes, motores, ferramentas e pastas aparecem juntos. Os discos ausentes continuam visíveis.

15.2 Mapa guardado e atualização

O mapa privado abre imediatamente. O arranque e as visitas atualizam o estado sem repetir pesquisas completas ou medir tamanhos. Atualizar pesquisa discos montados e mede pastas. Após a primeira pesquisa, os novos discos são pesquisados uma vez. Com esta página visível, as verificações de montagem respeitam um mínimo de 15 segundos, o intervalo de fundo, o modo manual e a pausa de foco. As verificações de versão exigem confirmação.

Se um disco foi removido definitivamente, selecione o nó cinzento e Esquecer este disco. A confirmação remove apenas entradas guardadas do mapa, não ficheiros nem definições. As pastas configuradas podem reaparecer ao atualizar. Os motores encontrados mostram separadamente o caminho de arranque (incluindo current) e a compilação resolvida. Os tamanhos seguem os destinos conhecidos dos lançadores; tamanhos não atribuíveis ficam ocultos. Com um perfil de definições explícito, system-map.json fica junto a settings.json. O mapa predefinido usa a cache da aplicação. Os novos registos persistentes de Desfazer ficam sempre em model-moves/ junto a settings.json; os registos antigos na cache predefinida continuam legíveis. Guarde-os até desfazer as respetivas transferências.

15.3 Ligar, mover e desfazer

Selecione um componente para Verificar, Voltar a ligar ou Usar pasta. Arraste um motor para uma pasta de modelos para pré-visualizar a ligação, ou uma pasta para o motor no mapa de IA para configurar o caminho. As ligações confirmadas guardam configuração e cópias sem iniciar serviços. Arraste uma pasta GGUF para um disco para pré-visualizar a transferência. Leitores ativos bloqueiam a operação; entre discos, copia-se e verifica-se primeiro. Ligações e referências suportadas são atualizadas. Desfazer protege ficheiros novos. Lançadores personalizados e armazenamentos não suportados exigem um plano manual ou de IA revisto. As sugestões de montagem são texto: fstab nunca é alterado.

15.4 Análise e ajuda de IA

Estado do hardware, orientação GPU e resultados aparecem nos nós. A verificação com IA exige consentimento ao resumo e mensagem exatos: o agente e o serviço do modelo online recebem caminhos e podem consumir tokens. Corrigir e simplificar abrem tarefas protegidas para revisão. Não

são incluídas credenciais. Utilização no mapa de IA mostra execuções e tokens. Reveja duplicados e serviços sem uso antes de consolidar; mantenha cópias.

CAPÍTULO 16

Verificação de saúde e segurança

Um agente de programação lê e escreve ficheiros e executa comandos no seu computador. Quanto pode fazer *sem lhe perguntar* está nas suas próprias definições, com palavras diferentes em cada agente. A **Verificação de saúde e segurança** junta tudo isto: na página de detalhes de cada agente e de cada LLM local, quatro cartões mostram se funciona, o que pode fazer, o que isso significa em palavras simples e o que reduz o risco.

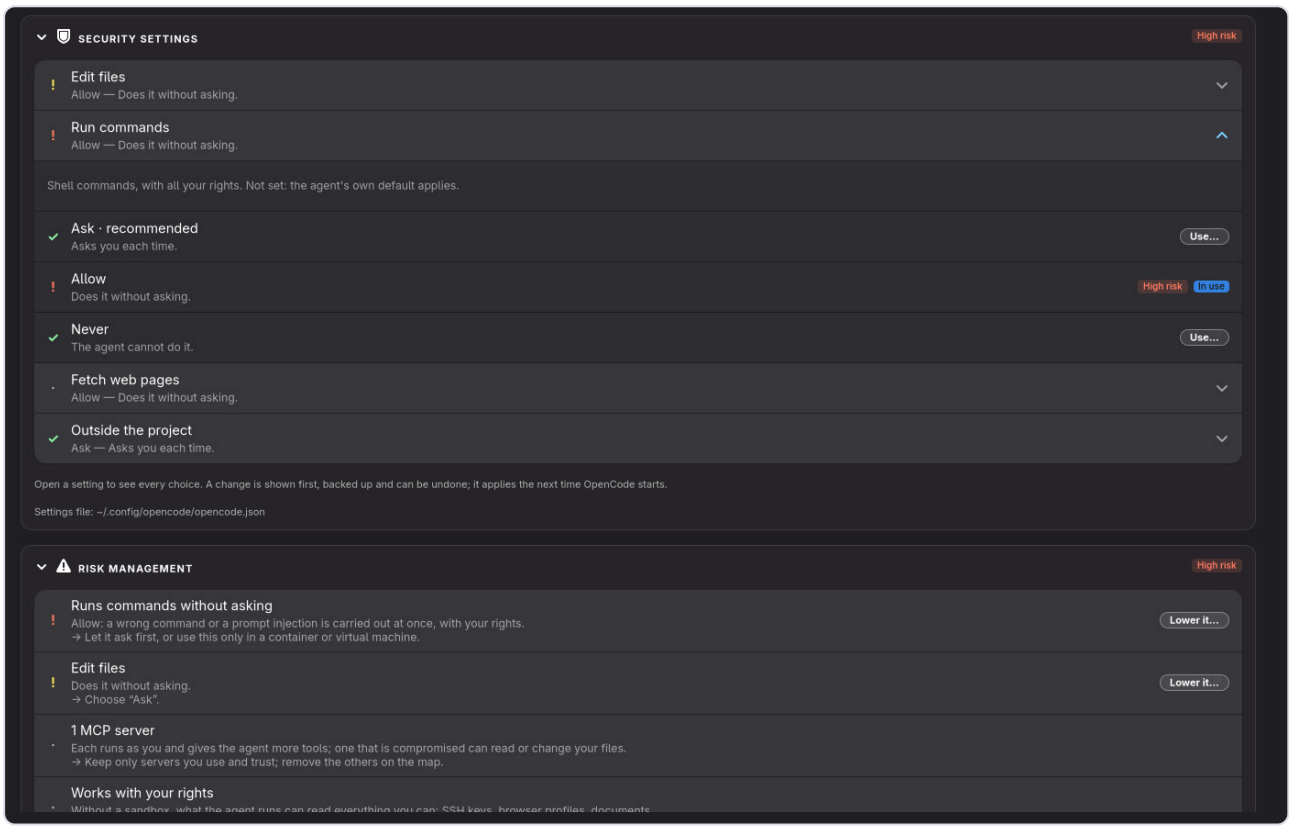


Figura 14 · As definições de segurança do OpenCode com **Executar comandos** aberto, e o início da sua gestão de riscos.

16.1 Ler os cartões

Os distintivos dos cartões recolhidos resumem tudo: **Saudável**, **Precisa de atenção** ou **Não configurado** em Estado, e **Risco baixo**, **Risco médio** ou **Risco alto** em Definições de segurança e Gestão de riscos. Abra um cartão para ver os pormenores:

Cartão	O que mostra
Estado	Cada verificação da configuração com um ✓, ou o que fazer a seguir: instalado, com sessão iniciada, o seu modelo e LLM local, servidores MCP e skills.
Definições de segurança	As definições do agente que decidem o que pode fazer sozinho, cada uma com o valor atual e uma frase sobre o que significa. O sinal à frente indica o risco: ✓ seguro, · baixo, um **!** amarelo médio, um **!** vermelho alto.

Cartão	O que mostra
Gestão de riscos	O que as definições significam em conjunto, o mais sério primeiro, cada um com o motivo e → o que o reduz. As proteções existentes aparecem com um ✓.
O que significam as palavras	Sandbox, aprovação, regras de permitir e negar e as outras palavras dos cartões, explicadas (ver As palavras explicadas).

As regras de permitir e negar e as pastas de confiança do agente (por exemplo a lista `permissions.allow` do Claude Code) aparecem por baixo das definições, uma regra por linha.

16.2 Alterar uma definição

1. Clique numa definição para a abrir. Vê o que controla e cada escolha com o seu risco; a que está em uso tem a marca **Em uso**, a escolha mais segura **recomendado**.
2. Clique em **Usar …** junto a uma escolha ou em **Reduzir …** junto a um risco. Uma página mostra a alteração em palavras (antes → depois), o risco antes e depois, e **O que muda**: as linhas do ficheiro de definições do agente.
3. Clique em **Alterar**. O ficheiro é copiado em segurança e escrito; uma notificação oferece **Desfazer** durante 30 segundos. O agente usa a nova definição no próximo arranque.

> **NOTA** Só essa chave muda; os comentários e tudo o resto no ficheiro ficam como estão. Uma definição que precisa de mais de uma linha num ficheiro — como o Hermes a executar os seus comandos em Docker — é mostrada e explicada, com onde a alterar, mas não é alterada daqui.

16.3 A segurança na pré-visualização Flutter

Na pré-visualização Flutter, os detalhes de um agente instalado têm **Definições de segurança**. O cartão mostra o distintivo de risco, cada definição com as suas opções numa lista (um ponto colorido indica o risco de cada opção, o valor predefinido mais seguro diz **recomendado**; uma definição que o ficheiro não nomeia indica que se aplica o valor predefinido do próprio agente), as regras de permitir e negar, o ficheiro de definições e a **Gestão de riscos** com **Reduzir…** onde uma definição reduz um risco. Escolher outro valor ou **Reduzir…** abre a **Pré-visualização** que uma alteração de ligação usa: as linhas do ficheiro de definições que mudam. **Aplicar** faz uma cópia do ficheiro e escreve-o, com **Desfazer** depois; só essa chave muda, os comentários e o resto do ficheiro ficam como estavam.

16.4 O estado na pré-visualização Flutter

Na pré-visualização Flutter, os detalhes de um agente instalado e cada LLM local em **Fornecedores e modelos** têm um cartão **Estado**: **Saudável**, **Precisa de atenção** ou **Não configurado**, uma frase e cada verificação — instalado, com sessão iniciada (só se vê se há um ficheiro de acesso ou uma chave), modelo, os seus LLM locais, ligações que falharam, servidores MCP e skills; num LLM local, se corre, os seus modelos e os agentes ligados. Uma verificação que não está bem diz o que fazer a seguir, com um comando para **Copiar** quando existe. Abre-se sozinho quando há algo a fazer.

16.5 O que é verificado em cada agente

Agente	Definições mostradas e alteráveis	Ficheiro
Claude Code	Modo de permissões (Perguntar primeiro, Só planear, Só o permitido, Editar sem perguntar, Revisão automática, Saltar todas as verificações), sandbox para comandos, bloquear o modo de contorno; regras permitir/perguntar/negar	`~/.claude/settings.json`
Codex	Quando o Codex pergunta, sandbox (só leitura, pasta do projeto, nenhuma), rede na sandbox; pastas de confiança	`~/.codex/config.toml`
GitHub Copilot	Modo de permissões (Perguntar primeiro, Revisão automática, Permitir tudo); pastas de confiança, sites sem perguntar	`~/.copilot/settings.json`
Cursor CLI	Aprovação, sandbox, rede na sandbox, pesquisa web sem perguntar; regras permitir/negar	`~/.config/cursor/cli-config.json`
Hermes	Comandos perigosos (perguntar sempre, revisão automática, nunca perguntar), ocultar segredos; onde correm os comandos (só mostrado)	`~/.hermes/config.yaml`
OpenCode	Perguntar, permitir ou nunca para editar ficheiros, executar comandos, obter páginas web e trabalhar fora do projeto	`~/.config/opencode/opencode.json`
Pi	As suas ferramentas: ler, escrever e executar comandos, ou só ler. O Pi, por conceção, nunca pergunta, por isso as suas ferramentas são as suas permissões.	`~/.pi/agent/settings.json`

Para outros agentes, o cartão mostra o seu estado e os seus servidores MCP e diz onde procurar nas próprias definições do agente. Para um **LLM local** mostra quem o pode alcançar: só este PC (`127.0.0.1` , como deve ser) ou qualquer dispositivo da sua rede.

16.6 Controlar o risco

O nível de risco é o ponto mais sério que o cartão encontra:

- **Risco alto** — pode fazer coisas sérias sem perguntar: executar qualquer comando, ou trabalhar sem sandbox e sem perguntas. Aceitável num contentor descartável ou numa máquina virtual; no seu próprio PC, faça-o perguntar primeiro.
- **Risco médio** — algumas coisas acontecem sem perguntar, como editar ficheiros do projeto, ou uma revisão automática decide por si.
- **Risco baixo** — pergunta antes de tudo o que importa, ou não o pode fazer de todo.

Bons hábitos, sejam quais forem as definições:

- **Perguntar primeiro nos comandos** no seu próprio computador; permita comandos isolados e inofensivos (como `git status`) com uma regra de permitir em vez da shell inteira.
- **Ligue a sandbox** onde o agente tiver uma (Claude Code, Codex, Cursor CLI) e deixe a rede lá dentro desligada a não ser que uma tarefa precise dela.
- **Inicie os agentes na pasta do projeto**, não na sua pasta pessoal, e mantenha chaves e palavras-passe longe.
- **Poucos servidores MCP**: só os que usa e em que confia. Cada um corre com as suas permissões.
- **O trabalho sem supervisão** (modo de contorno, «YOLO», Pi com todas as ferramentas) deve correr num contentor ou numa máquina virtual.
- **LLMs locais em `127.0.0.1`**: os seus servidores não têm palavra-passe.

> **ATENÇÃO** Uma revisão automática (o modo *auto* do Claude Code, *assisted* do Copilot, *auto-review* do Cursor, *smart* do Hermes) é uma segunda IA que julga cada passo. Poupa perguntas, mas é um juízo, não uma muralha.

16.7 As palavras explicadas

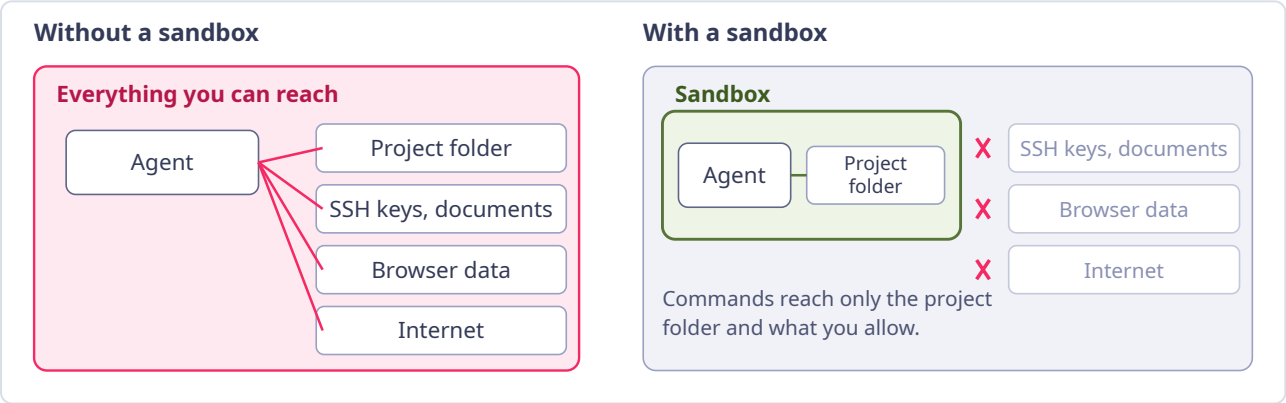


Figura 15 · O que muda uma sandbox: os comandos só alcançam a pasta do projeto e o que permitir.

Sandbox	Um espaço isolado onde correm os comandos do agente. Lá dentro, um comando só pode tocar na pasta do projeto (e no que permitir) — não no resto da sua pasta pessoal, nas suas chaves SSH ou nos dados do navegador, e muitas vezes nem na internet. Sem sandbox, um comando do agente pode fazer tudo o que o utilizador pode.
Aprovação (perguntar primeiro)	Antes de o agente editar um ficheiro ou executar um comando, mostra o que quer fazer e espera pelo seu sim ou não. Quanto menos puder fazer sem perguntar, mais controlo mantém — e mais vezes lhe pergunta.
Regras de permitir e negar	Listas de comandos ou ferramentas que o agente pode usar sem perguntar (permitir) ou nunca (negar). Negar ganha a permitir. Uma regra que permite qualquer comando de shell desliga a aprovação para a shell.
Pasta de confiança	Uma pasta em que disse que o agente pode trabalhar. Lá, usa também as próprias instruções e definições da pasta, que podem vir de outra pessoa quando clona um projeto.
Acesso à rede	Se os comandos podem alcançar a internet. Código que lê os seus ficheiros e pode enviar dados poderia divulgá-los; instalar pacotes também precisa da rede.
Contorno («YOLO»)	Tudo sem perguntar e sem limites. Só para uma VM ou um contentor descartáveis.
Revisão automática	Um segundo modelo de IA deixa passar as ações inofensivas e pergunta pelas restantes.
Injeção de prompts	Instruções escondidas numa página web, num ficheiro ou na resposta de uma ferramenta, que tentam levar o agente a fazer algo que não pediu. Perguntar primeiro, uma sandbox e nenhuma rede travam-nas.
Ocultar segredos	Texto que parece uma palavra-passe, um token ou uma chave é mascarado antes de o modelo o ver ou de ir parar a um registo.
Ferramentas	O que o agente pode fazer de todo: ler, escrever, executar comandos, pesquisar na web. Uma ferramenta que não tem não pode ser mal usada.
Servidor MCP	Um pequeno programa que dá ferramentas extra ao agente. Corre como o utilizador, com as suas permissões.
127.0.0.1 (localhost)	Um endereço que só os programas deste PC alcançam. Um servidor em `0.0.0.0` é alcançável a partir de qualquer dispositivo da sua rede.

CAPÍTULO 17

Complementos

Tudo o que o Maprisma entende de um agente ou de um fornecedor de LLM é um **adaptador**: como é encontrado e instalado, as suas sessões, definições de segurança, servidores MCP e skills. No Maprisma, cada adaptador é um **complemento**: os pacotes trazem-nos todos, assinados e desativados, e o utilizador ativa os que usa. Os complementos também trazem agentes novos, fornecedores novos e funções que vão além dos números, como o **Insights** (tópicos das sessões e um resumo do Claude). O plugin AI Agents Map tem os adaptadores integrados e não tem complementos.

17.1 Configurar o Maprisma

Um Maprisma novo abre na sua **página de boas-vindas**. Cada agente e LLM local que o Maprisma conhece é um cartão com o seu próprio ícone, em três grupos: **Agentes de IA**, **LLMs locais** e **Fornecedores online** (os nomes da Anthropic, da OpenAI e dos outros no mapa). Cada cartão diz se o programa está aqui:

- **Instalado** (verde) — encontrado da mesma forma que o mapa o encontra; o cartão já está assinalado;
- **Usado antes** — agora não instalado, mas as suas definições ainda lá estão;
- **Não instalado** — assinale-o para o instalar depois a partir do seu cartão.

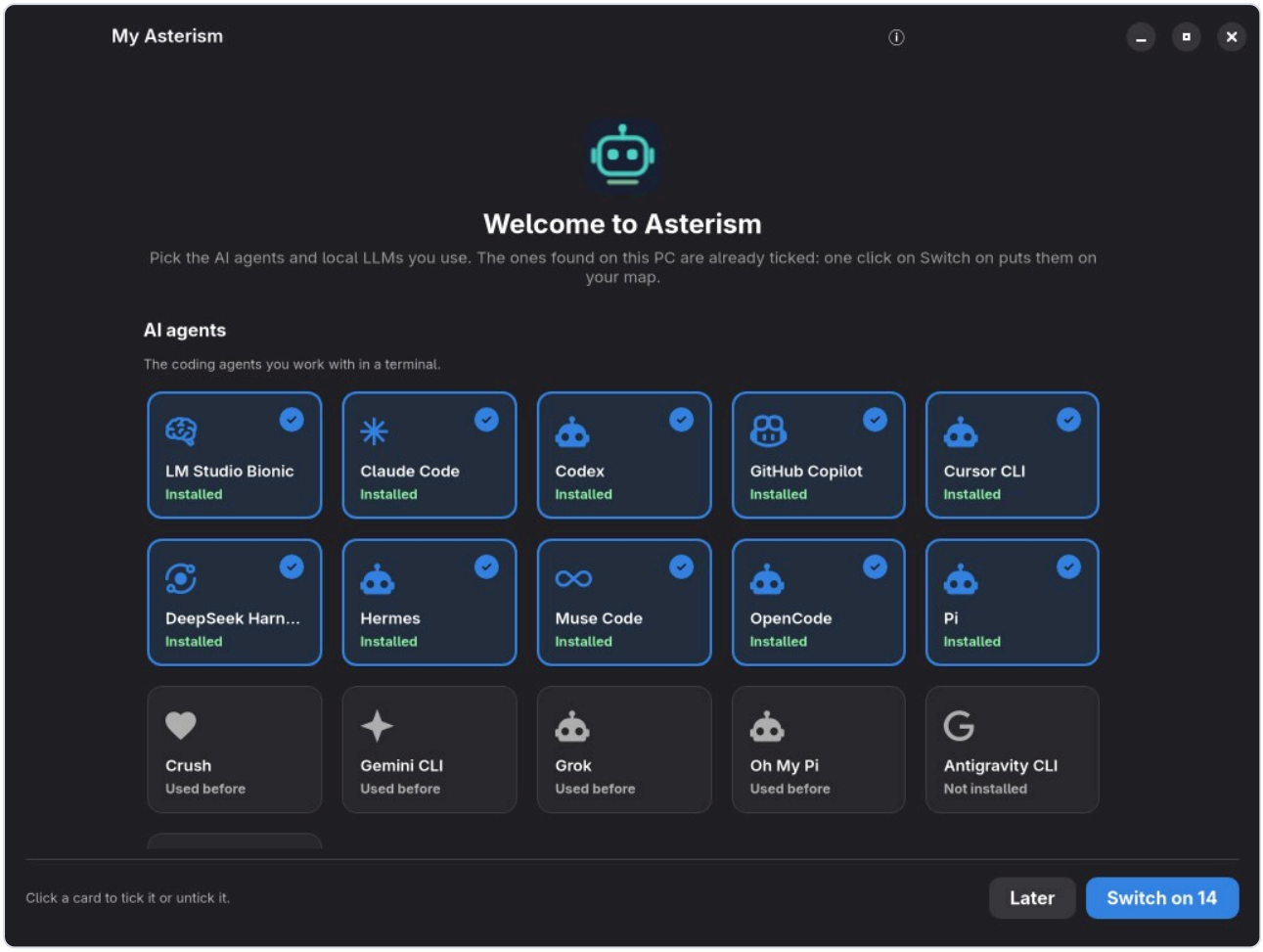


Figura 16 · A página de boas-vindas: os programas encontrados vêm primeiro e estão assinalados.

Clique num cartão (ou prima `Espaço` sobre ele) para o assinalar ou desassinalar; um cartão assinalado tem contorno azul e um visto. **Ativar** (com o número de cartões assinalados) carrega-os e mostra o mapa; se assinalou programas que não estão instalados, abre-se **Configuração**: ··· nos seus cartões › **Instalar...**. **Mais tarde** começa com um mapa vazio. Enquanto nada estiver ativado, o Maprisma volta a abrir na página de boas-vindas em cada arranque. Depois, os mesmos cartões estão em **Configuração** (o botão do cabeçalho), onde agentes e fornecedores se ativam e desativam a qualquer momento.

17.2 A página de complementos

Abra-a com o botão da peça de puzzle no cabeçalho, a partir de Definições › **Complementos** ou de qualquer sugestão «complemento disponível». Mostra, em cartões de grupo recolhíveis que começam abertos:

- **Sugeridos para este PC** — um complemento cujo agente ou fornecedor está instalado aqui mas ainda não é mostrado;
- **Complementos instalados** — ativado, desativado ou *não carregado* com o motivo (por exemplo um ficheiro alterado); agentes e fornecedores separados por uma linha;
- **Disponíveis** — os restantes complementos do catálogo;

- **Incluído no Maprisma** — agentes e fornecedores que nenhum complemento instalado traz (quando corre o Maprisma a partir da sua pasta de código sem complementos).

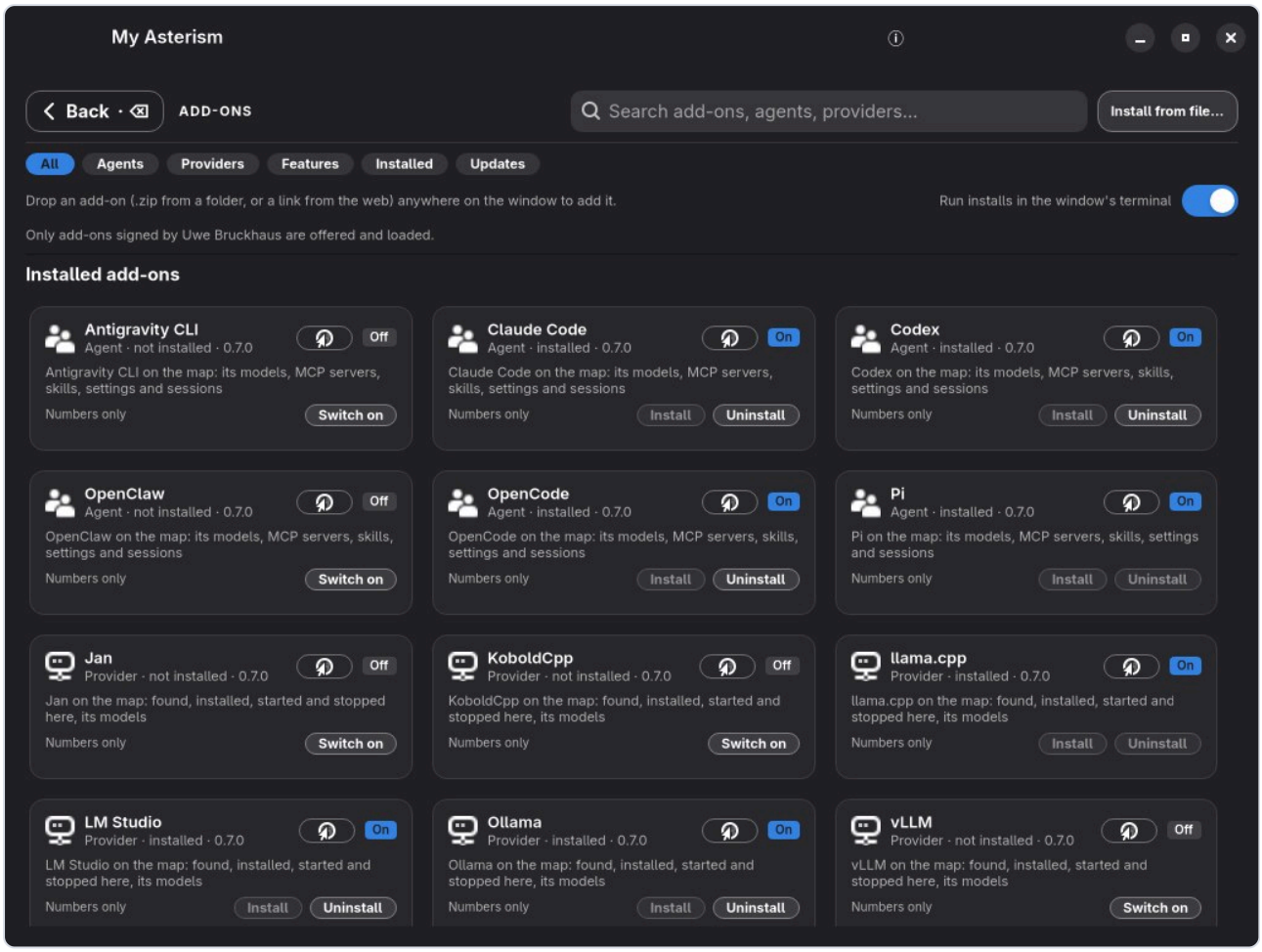


Figura 17 · Complementos instalados: ativados ou desativados, com Instalar e Desinstalar para o programa.

O cartão de um agente ou LLM local ativado tem **Instalar** e **Desinstalar** para o próprio programa (o que não se aplica fica a cinzento), e cada cartão com página inicial tem um botão com um globo que a abre no seu navegador.

A pesquisa encontra nomes, descrições e o que um complemento lê; os filtros por cima reduzem a lista a agentes, fornecedores, funções, aos instalados ou aos que têm atualização. Clique num cartão para ver os detalhes: o que lê, o que envia, o que pode fazer e o seu nível de privacidade.

17.3 Instalar, ativar e desativar, atualizar, remover

Instalar (ou **Obter complemento**) mostra primeiro o que o complemento lê, envia e pode fazer; só depois do seu OK é descarregado, verificado e ativado. **Desativar** e **Ativar** atuam de imediato, sem reiniciar: os seus agentes, fornecedores, secções e cartões aparecem ou desaparecem em todo o lado. **Atualizar** substitui-o pela versão mais recente do catálogo depois de verificar a nova; **Remover** move-o para o lixo. **Instalar a partir de ficheiro...** aceita um zip de complemento que tenha descarregado; aplicam-se as mesmas verificações.

Quando o catálogo tem versões novas de complementos instalados, os cartões da **Configuração** dizem *Atualização do complemento*, a página de problemas em aberto (o **!** vermelho no cabeçalho) mostra **Atualizações de complementos prontas** com **Atualizar tudo** e **Complementos**, e a página de complementos mostra uma barra com **Atualizar tudo**. O menu ··· do cartão (ou um clique direito nele) tem também **Atualizar o complemento para** *versão*, só para esse complemento. Atualizar tudo atualiza-os um a um, com as mesmas verificações, e diz numa só mensagem quais foram atualizados. Só corre uma instalação ou atualização de cada vez; outra iniciada entretanto pede para tentar de novo daqui a pouco.

> **SÓ DO AUTOR.** O Maprisma só carrega complementos assinados por Uwe Bruckhaus (impressão digital da chave `SHA256:e5cC634Ke3Our5LAeSc88qs22NKZEHTncBW5Ig3amWQ`). Antes de correr o que quer que seja de um complemento, a assinatura é verificada com o `ssh-keygen` do OpenSSH, cada ficheiro é comparado com a sua soma SHA-256 e nenhum ficheiro pode ter sido acrescentado. Uma transferência tem ainda de corresponder à soma SHA-256 do catálogo assinado.

17.4 Níveis de privacidade

Nível	O que o complemento pode ler ou enviar
Só números	Contagens, tempos, tokens, linhas alteradas, custo — o que o Maprisma mostra sem complementos.
Títulos de sessão	Os títulos que um agente deu ele próprio às suas sessões (nunca as conversas).
Títulos e um resumo do Claude	Esses títulos e números, enviados para o seu próprio Claude Code — só com um clique, após uma pré-visualização exata do que é enviado.

Um complemento nunca pode fazer mais do que o seu nível permite. Adicionar um agente ou um fornecedor, ou instalar um programa, são permissões à parte que o seu cartão indica.

17.5 Tipos de complementos

Tipo	O que traz
Agente	Um agente de programação: onde guarda as definições, sessões, servidores MCP e skills, como se instala e como inicia sessão. Tudo o resto o Maprisma constrói a partir disto — o cartão em Configuração, a sua caixa no mapa, Uso e tokens, Projetos, Estado e Segurança.
Fornecedor	Um programa de LLM local ou os fornecedores online: como é encontrado e instalado e como os agentes o alcançam. O Maprisma constrói o seu cartão, a sua caixa no mapa e a sua página de detalhes — incluindo **Uso e tokens**: o que os seus agentes enviaram através dele, a partir dos próprios números de sessão.

Tipo	O que traz
Extensão	Mais dados sobre um agente ou fornecedor que outro complemento traz, sem o substituir — por exemplo o seu plano e limites, ou um cartão na sua página de detalhes. Só preenche o que o complemento do agente deixa vazio; a página do complemento diz o que acrescenta e o que disso não é usado.
Função	Algo que abrange vários agentes, como o Insights (o cartão Tópicos).

A secção de um complemento na página de detalhes de um agente ou LLM local vai para o cartão de que trata: Uso e tokens, Estado, Tópicos (só nos agentes) ou um cartão próprio com o seu nome.

Um complemento de agente descreve o seu agente como dados (um ficheiro chamado `agent.toml`); a maioria não traz código nenhum, e a página do complemento di-lo: *Não traz código* ou, para os poucos agentes que precisam de algo (por exemplo a sua própria base de dados de sessões), *Dados sobre o agente, e um pouco de código para o que os dados não conseguem dizer*.

17.6 Um agente novo no seu computador

Quando instala um agente que o Maprisma ainda não conhece, ele procura no catálogo (Definições › **Procurar complementos online**, ligado por omissão: a lista é lida do GitHub de poucas em poucas horas e só o pedido é enviado). Se um complemento conhece esse agente, **Atualizações e sugestões** (o **!** do cabeçalho) di-lo (*«O Kilo Code está neste PC mas não no mapa»*), e aparece um cartão em **Sugeridos para este PC**; um clique instala-o. Se ainda não houver complemento, os detalhes do agente dizem-no, com uma ligação para pedir um.

17.7 Criar um complemento

Um novo LLM local ou um novo agente só precisa de um complemento: o Maprisma constrói o seu cartão, a sua caixa e as suas linhas no mapa, os seus detalhes, a página Modelos e a página Segurança a partir do que o complemento diz sobre ele. Se souber programar um pouco, comece pelos modelos no repositório do Maprisma (`templates/`, explicados em `docs/addons.md`): um ficheiro para um LLM local, um ficheiro de factos para um agente. Envie o complemento para `github.com/uBruckhaus/asterism-addons`; depois de revisto e assinado, aparece para todos em **Disponíveis**.

CAPÍTULO 18

Privacidade e segurança

- **Sem permissões de administrador para a app.** Nunca corre como root nem vê uma palavra-passe. Instalar um agente que vem como pacote do sistema (como o Hermes ou o OpenClaw) executa o gestor de pacotes do sistema com `sudo` num terminal que vê, depois de lhe mostrar o comando.
- **Nenhuma credencial lida.** Lê as definições dos agentes, mas nunca os seus ficheiros de credenciais. Se um agente tem sessão iniciada percebe-se pela existência desse ficheiro. Os valores de chaves e cabeçalhos nunca são mostrados.
- **Nenhuma conversa lida.** Os números de uso são lidos apenas como números e tempos. A única exceção é acompanhar uma instância *Em segundo plano* (Terminais…), com um clique seu, só para essa instância; nada disso é guardado.
- **Rede só quando a pede** — pesquisar ou instalar modelos, servidores MCP ou skills, testar um servidor —, mais uma verificação de atualizações de poucas em poucas horas.
- **Transferências verificadas.** Os programas só são instalados se a transferência corresponder à soma publicada pela sua versão.
- **Pré-visualização e Desfazer.** Cada alteração às definições de outro programa é mostrada antes, copiada em segurança e pode ser desfeita. As definições que podem conter chaves são escritas legíveis só pelo utilizador.
- **Limitado.** Cada programa corre sem shell, e cada resposta é lida com limite de tamanho e de tempo.
- **Funções de IA só quando as escolhe.** As pesquisas do Smart Check correm com um clique, as verificações IA diárias só quando estão ligadas. O agente recebe um pequeno resumo (nomes, versões, modelos), nunca chaves nem valores de definições, entregue de forma que outros utilizadores do computador não conseguem ler; as suas respostas são verificadas antes de serem mostradas e nunca são executadas. As tarefas correm num terminal que vê, e o agente pergunta antes de cada passo. Usa-as por sua conta e risco.

O README de cada repositório lista por completo o que é lido, iniciado e escrito.

CAPÍTULO 19

Resolução de problemas

Problema	O que fazer
O mapa está apertado ou cortado.	A janela tem menos de cerca de 1300 píxeis. Alargue a janela, ou reduza o zoom (`Ctrl`+`-`).
Falta no mapa uma caixa que espero.	O mapa só mostra o que está configurado: veja em **Configuração** (um complemento desativado, um servidor MCP que nenhum agente usa) e nos interruptores **Mostrar**.
Iniciar aqui falta ou não faz nada.	Instale o VTE para GTK 4 (`vte4`), ou use **Iniciar no terminal**.
Uma linha está vermelha tracejada.	Clique nela para ver sugestões, corrija o que dizem e depois **Testar de novo**. Ver Quando uma ligação falha.
O LLM local não arranca com o agente.	Verifique Definições › **Iniciar LLMs locais com os seus agentes** e se o agente está ligado no mapa. `systemctl --user status ai-agents-map-providers` mostra o vigilante.
A luz colorida de um LLM local até ao seu modelo não aparece.	Só aparece enquanto um programa que não está no mapa gera com esse LLM, e apaga-se cerca de dez segundos depois do seu último pedido. Verifique se o mapa está visível (uma janela larga), se o programa envia mesmo algo e se Definições › **Animações do mapa** está em **Agentes e outros programas**.
O agente responde mal ou corta-se com um modelo local.	O contexto pode ser pequeno demais. Abra o modelo em **Modelos… › Instalados** e use **Definir contexto** com o melhor valor para a sua placa.
Uma transferência parou.	Depois de uma quebra de ligação continua sozinha. Depois de reiniciar, use **Continuar** na faixa do topo.
Quero desfazer uma alteração anterior.	As cópias de segurança estão em `~/.local/state/ai-agents-map/backups/` .

APÊNDICE

Referência

A.1 Atalhos de teclado

`F1`	Ajuda
`Ctrl`+`,`	Definições
`F5`	Atualizar (o mapa e Configuração)
`Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0`	Ampliar o mapa, reduzi-lo, voltar a 100 %
`Ctrl`+roda	Zoom do mapa
`Shift`+clique	Acrescentar à seleção ou retirar dela
`Esc`	Limpar a seleção; voltar de uma página
`Retrocesso`	Voltar de uma página

A.2 Uso com o teclado na pré-visualização Flutter

Os atalhos acima também funcionam na pré-visualização Flutter: `F1`, `Ctrl`+`,`, `F5`, `Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0` para o mapa visível, e `Esc` ou `Retrocesso` para voltar de uma página. Num campo de texto `Retrocesso` apaga, uma caixa de diálogo fecha com `Esc` e um terminal mantém ambas as teclas para o seu agente.

`Tab` percorre tudo o que está no ecrã, `Enter` ou `Espaço` usa-o. No mapa, `Enter` sobre uma caixa abre-a, e o seu botão ⋮ abre as suas ações. No editor do mapa cada arrasto tem um caminho pelo teclado: **Ações** › **Ligar** de uma caixa, depois `Tab` até à caixa de destino e `Enter` (`Esc` cancela); **Mover ligação**, **Mover para cima** e **Mover para baixo** estão no mesmo menu. A aplicação é verificada quanto a controlos com rótulo, alvos de pelo menos 24 × 24 píxeis e contraste de texto de pelo menos 4,5:1, em claro e escuro.

A.3 Ficheiros e pastas

Caminho	Conteúdo
`~/.config/maprisma/` MAPRISMA `~/.config/omarchy/ai-agents-map/` OMARCHY	Definições, a lista de servidores MCP, agentes pesquisados, ligações falhadas.
`~/.local/state/ai-agents-map/backups/`	Cópias de segurança de cada ficheiro que a app alterou.
`~/.local/state/ai-agents-map/jobs.json`	Transferências e instalações interrompidas.

Caminho	Conteúdo
`~/.local/state/ai-agents-map/reorganize.json`	O registo da última arrumação, para Desfazer.
`~/.cache/ai-agents-map/`	Resultados de pesquisa, listas de ferramentas, verificações de atualizações, o último Smart Check (`smart-check.json`) e as atualizações MCP das verificações IA (`mcp-updates.json`).
`~/.local/state/ai-agents-map/ai-help/`	O Protocolo: um ficheiro por cada execução de ajuda IA (as últimas 200).
`~/.local/state/omarchy-plugin-ai/runs.json`	As pesquisas e tarefas de IA desta app e de outros plugins, com os seus tokens: as estatísticas do Smart Check.
`~/.config/systemd/user/ai-agents-map-providers.service`	O vigilante que inicia os LLMs locais com os seus agentes.

A.4 Glossário

Aprovação Um agente pergunta antes de alterar um ficheiro ou executar um comando (ver As palavras explicadas).

Contexto Quantos tokens de texto um modelo consegue ter presentes de uma vez.

GGUF O formato de ficheiro de modelos usado pelo llama.cpp, LM Studio, Ollama e KoboldCpp.

Injeção de prompts Instruções escondidas numa página, num ficheiro ou na resposta de uma ferramenta que tentam conduzir um agente.

Quantização Guardar um modelo com menos bits por peso (por exemplo *Q4_K_M*, cerca de 4,8 bits) para que fique mais pequeno, com pouca perda de qualidade.

Mistura de especialistas Um modelo em que só uma parte dos parâmetros trabalha em cada token (*35B-A3B*: 35 mil milhões no total, 3 mil milhões ativos), o que o torna rápido para o seu tamanho.

Sandbox Um espaço isolado onde correm os comandos de um agente, que só alcançam a pasta do projeto e o que é permitido.

Token Um pedaço de texto, cerca de três quartos de uma palavra, que os modelos leem e escrevem.

Serviço de utilizador Um programa em segundo plano que o systemd inicia só para o seu utilizador, sem permissões de administrador.